

dr0idx

University Management System Database Design & Implementation Report

A thirteen-table normalised MariaDB schema, a Flask JSON API and a no-framework front end — with every constraint, index and query plan in this document verified by execution rather than by inspection.

Project	dr0idx — university portal for students and teachers
Repository	<code>github.com/0exu/dr0idx</code>
Database	MariaDB / MySQL — InnoDB, <code>utf8mb4_unicode_ci</code>
Schema name	<code>login</code> — 13 tables, 17 foreign keys, 2 views
Application tier	Python 3 · Flask · native <code>mariadb</code> connector
Presentation tier	HTML5 · CSS3 · vanilla JavaScript, no build step
Design source	<code>ER_MODEL_CONTINUE.drawio</code> , <code>mariadb_mysql.sql</code>
Verification	Loaded into an isolated database; constraints exercised and <code>EXPLAIN</code> captured
Date	5 October 2026

Contents

- How to read this document
- 1 · Technology stack
- 2 · Running the system
- 3 · Application architecture
- 4 · The database layer
 - 4.1 Design objectives
 - 4.2 The ER model as drawn
 - 4.3 From drawing to schema
 - 4.4 The schema as built
 - 4.5 Complete data dictionary
 - 4.6 Keys, constraints and referential actions
 - 4.7 Functional dependencies and normalisation
 - 4.8 Indexing strategy
 - 4.9 Views
 - 4.10 The seed data
- 5 · Every endpoint, and the SQL behind it
 - 5.1 Endpoint summary
 - 5.2 The session guard
 - 5.3 Authentication
 - 5.4 Student reads
 - 5.5 Teacher reads and the write path
- 6 · Security design
 - 6.1 Password hashing
 - 6.2 Server-side sessions
 - 6.3 What the design gets right
 - 6.4 What a production deployment would still need
- 7 · Verified findings
 - 7.1 HIGH — `ON DELETE CASCADE` on the login keys points the wrong way
 - 7.2 HIGH — `ACADEMICS` and `ATTENDANCE` have no foreign key to `STUDENT`
 - 7.3 MEDIUM — `recent_notices` is a global count, not a per-student one
 - 7.4 MEDIUM — `ACADEMICS` ' primary key leads with the wrong column
 - 7.5 MEDIUM — the seed data leaves the 30-day attendance window stale
 - 7.6 MEDIUM — `excused` attendance is counted but never shown
 - 7.7 LOW — `new_materials` counts all 100 materials as “new this week”
 - 7.8 LOW — `NOTICES.EXPIRES_AT` can silently hide a notice forever
 - 7.9 LOW — cosmetic and consistency items
 - 7.10 Severity summary

- 8 · The application, screen by screen
 - 8.1 Role selection
 - 8.2 Login
 - 8.3 Student dashboard
 - 8.4 Study materials
 - 8.5 Academics
 - 8.6 Attendance
 - 8.7 Notices
 - 8.8 Teacher dashboard
 - 8.9 My courses
 - 8.10 Teacher academics
 - 8.11 Teacher attendance
 - 8.12 Notice creator
- 9 · What to build next
 - 9.1 Correctness — do these first
 - 9.2 Schema evolution the screens already imply
 - 9.3 Hardening
 - 9.4 Modelling debt, if there is time
- 10 · Operations
 - 10.1 Connecting by hand
 - 10.2 Introspection queries
 - 10.3 Troubleshooting
 - 10.4 Backup and reset
- Appendix A · Endpoint reference
- Appendix B · The DDL as shipped
- Appendix C · Reproducing the verification

How to read this document

This is a **database** document that happens to describe a web application. The front end is discussed only where it changes what the database must store or return.

Every factual claim in the data sections carries a verification note. Where the repository contains a claim that does not survive execution, it is reported as a finding rather than repeated as fact. Section 7 lists all of them together.

Source of truth. Where this report and the repository disagree, the repository wins: `plan-files/ER_MODEL_CONTINUE.drawio` for the conceptual model, `plan-files/mariadb_mysql.sql` for the DDL, `plan-files/seed_data.sql` for the data, and `web-content/backend/server.py` for every statement quoted in Section 5. Appendix B reproduces the DDL for convenience, but it is a copy.

VERIFIED

Verification method. The DDL in `plan-files/mariadb_mysql.sql` and the seed data in `plan-files/seed_data.sql` were loaded into a throwaway database (`dr0idx_verify`) on MariaDB 13.0.2. Column types, index definitions, foreign keys, referential actions and check constraints were read back from `information_schema` . Execution plans come from `EXPLAIN` . Constraint behaviour was confirmed by deliberately violating each constraint and recording the error the server raised. The repository database was never modified.

1 · Technology stack

The stack is fixed by `README.md` and realised exactly as written.

Layer	Technology	Where it lives
Presentation	HTML5, CSS3, vanilla JavaScript	<code>web-content/**/*.html</code> , <code>*.css</code> , <code>*.js</code>
Application	Python 3, Flask, Flask-CORS	<code>web-content/backend/server.py</code>
Connectivity	Native <code>mariadb</code> driver (C-based, not an ORM)	<code>web-content/backend/server.py</code>
Configuration	<code>python-dotenv</code>	<code>web-content/backend/.env</code>
Data	MariaDB / MySQL, InnoDB	<code>plan-files/mariadb_mysql.sql</code>
Version control	git on GitHub	repository root
Design source	draw.io	<code>plan-files/ER_MODEL_CONTINUE.drawio</code>

1.1 Repository map

```

dr0idx/
├── .env.example          template for backend credentials
├── README.md            stack declaration + local runbook
├── plan-files/          design & planning (this document lives here)
│   ├── ER_MODEL_CONTINUE.drawio hand-drawn ER model – the design source
│   ├── mariadb_mysql.sql  authoritative DDL: 13 tables, 17 FKs, 2 views
│   ├── seed_data.sql      deterministic demo dataset
│   ├── HOW-TO-RUN-SQL.txt two-line loader
│   ├── demo.md            original four-step user journey
│   └── PROJECT-REPORT.pdf this document
├── web-content/         the served document root
│   ├── index.html script.js style.css role selector (landing page)
│   ├── s_login/         student login + student dashboard
│   ├── t_login/         teacher login + teacher dashboard
│   ├── backend/         server.py, requirement.txt, .env
│   ├── image-sources/    logo and wallpaper assets
│   └── output-files/     project video and screen captures

```

1.2 Front-end routing

The front end has no router and no framework. Routing is the URL hash.

Screen	File	Hash sections
Role selector	<code>web-content/index.html</code>	—
Student login	<code>web-content/s_login/s_login.html</code>	—
Student dashboard	<code>web-content/s_login/student/index.html</code>	<code>#home</code> , <code>#study-materials</code> , <code>#academics</code> , <code>#attendance</code> , <code>#notices</code>
Teacher login	<code>web-content/t_login/t_login.html</code>	—
Teacher dashboard	<code>web-content/t_login/teacher/index.html</code>	<code>#home</code> , <code>#my-courses</code> , <code>#academics</code> , <code>#attendance</code> , <code>#notices</code>

A hash change fires `render()` in the relevant `script.js`, which reads `location.hash`, activates the matching `<section class="page">`, highlights the sidebar link and calls `loadPage(id)`. Each section fetches its data **once per visit** — a `loaded` Set short-circuits repeat navigation, which is why no `ORDER BY` needs to be stable across re-fetches.

1.3 Session handling on the client

Three `sessionStorage` keys are set on successful login and cleared on logout:

Key	Student	Teacher
<code>*_auth</code>	<code>'1'</code>	<code>'1'</code>
<code>*_roll / *_tid</code>	roll number	teacher ID
<code>*_session</code>	64-hex session token	64-hex session token

Two guards are worth noting because they are database-relevant. `student/index.html` contains an inline `<script>` in `<head>` that redirects before first paint if `student_auth !== '1'`, and both dashboard scripts re-run the check on the `pageshow` event when `event.persisted` is true, closing the back/forward-cache hole. **Neither guard talks to the database.** The browser flag is cosmetic; the real gate is the `X-Session-Id / session_id` token, which every endpoint re-validates against `SESSIONS` on the server. See Section 6.

2 · Running the system

The runbook below is the working sequence, expanded from `README.md` and `plan-files/HOW-T0-RUN-SQL.txt` with the missing ordering made explicit.

The ordering matters. The schema must exist before the seed data, and the database service must be up before Flask starts. `HOW-T0-RUN-SQL.txt` states this in two lines; below is the same thing with the reasoning attached.

```

23:01:27 kitty
sudo mariadb -u corr ~/P/D/h/d/...

Os ~/output-files P main ?
..
Os ~/dr0idx P main ?
ls
drwxr-xr-x - omswastra 4 Oct 23:00 output-files
drwxr-xr-x - omswastra 4 Oct 04:00 plan-files
drwxr-xr-x - omswastra 2 Oct 01:27 web-content
-rw-r--r-- 1.5k omswastra 25 Sep 01:00 LICENSE
-rw-r--r-- 1.8k omswastra 1 Oct 21:41 README.md

Os ~/dr0idx P main ?
z web-content/

Os ~/web-content P main ? v20.10.0
ls
drwxr-xr-x - omswastra 4 Oct 03:20 backend
drwxr-xr-x - omswastra 2 Oct 01:21 image-sources
drwxr-xr-x - omswastra 4 Oct 02:17 s_login
drwxr-xr-x - omswastra 2 Oct 01:27 t_login
drwxr-xr-x - omswastra 29 Sep 01:22 venv
-rw-r--r-- 1.1k omswastra 2 Oct 01:22 index.html
-rw-r--r-- 829 omswastra 28 Sep 13:36 script.js
-rw-r--r-- 1.7k omswastra 2 Oct 01:27 style.css

Os ~/web-content P main ? v20.10.0
source venv/bin/activate.fish

Os ~/web-content P main ? v20.10.0 v3.14.7 venv
python3 backend/server.py
* Serving Flask app 'server'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI s
erver instead.
* Running on http://127.0.0.1:8080
Press CTRL-C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 694-878-527

MariaDB [login]> SHOW TABLES;
+-----+
| Tables_in_login |
+-----+
| ACADEMICS        |
| ASSESSMENTS      |
| ATTENDANCE        |
| COURSES           |
| NOTICES           |
| SESSIONS          |
| STUDENT           |
| STUDENT_LOGIN    |
| STUDY_MATERIALS  |
| STUDY_PROGRESS   |
| STU_COURSES       |
| TEACHER           |
| TEACHER_LOGIN    |
| v_student_dashboard |
| v_teacher_dashboard |
+-----+
15 rows in set (0.000 sec)

MariaDB [login]>

```

Figure 1 — Loading the schema, seeding data, and starting the Flask server

2.1 Load the database

```

# 1. schema first – creates the `login` database, 13 tables, 17 FKs, 2 views
mariadb -u root -p < plan-files/mariadb_mysql.sql

# 2. seed second – 9,265 rows. This file opens with `USE login;`
mariadb -u root -p login < plan-files/seed_data.sql

```

`mariadb_mysql.sql` is idempotent by construction: every table is `CREATE TABLE IF NOT EXISTS`, and each view is preceded by `DROP VIEW IF EXISTS` so the file can be replayed without error.

Verify the load with the statement block already at the end of `seed_data.sql`, which prints a row count per table. Expected totals:

Table	Rows	Table	Rows
STUDENT	50	ACADEMICS	750
STUDENT_LOGIN	50	ATTENDANCE	3,000
TEACHER	50	STUDY_MATERIALS	100
TEACHER_LOGIN	50	STUDY_PROGRESS	5,000

Table	Rows	Table	Rows
COURSES	5	NOTICES	20
STU_COURSES	250	ASSESSMENTS	15
SESSIONS	0	Total	9,265

NOTE

STUDENT_LOGIN may appear to hold 49 rows. `information_schema.TABLES` reports `TABLE_ROWS` from an InnoDB cardinality estimate, not a `COUNT(*)`. A freshly loaded table frequently reports one row fewer than it holds. The authoritative check is `SELECT COUNT(*) FROM STUDENT_LOGIN;`, which returns 50.

2.2 Configure the backend

```
cd web-content
python3 -m venv venv
source venv/bin/activate          # fish: venv/bin/activate.fish
pip install -r backend/requirement.txt
cp ../.env.example backend/.env
$EDITOR backend/.env              # fill in DB_USER, DB_PASSWORD, DB_NAME
```

`backend/requirement.txt` is four lines: `flask`, `flask_cors`, `mariadb`, `dotenv`. There is no ORM — the application issues SQL directly, which is why this document can trace every figure on screen back to a statement.

2.3 Start both tiers

```
# terminal 1 – static file server, document root
python3 -m http.server 5000

# terminal 2 – the API
systemctl start mariadb
python3 backend/server.py          # Flask on 127.0.0.1:8080
```

Then open <http://127.0.0.1:5000>.

Port note. The static server uses **5000**; the Flask API defaults to **8080**. The front-end JavaScript hardcodes `const API = 'http://localhost:8080'` in both dashboard scripts, so the API must be on 8080 unless you edit that constant. This is the single most common setup failure.

2.4 Demo credentials

Every seeded account shares one password per role.

Role	Identifier	Password	Example name
Student	CS001 ... CS050	stud123	CS001 = Aditya Sharma, SEM 3
Teacher	T001 ... T050	teach123	T001 = Aarav Sharma, Computer Science

T001 owns C001 Programming ; T013 owns C002 Discrete Math ; T025 owns C003 Reverse Engineering ;
T026 owns C004 Pentesting ; T041 owns C005 Soc .

2.5 Health check

```
curl http://localhost:8080/health  
# {"status":"Server is running"}
```

/health is the only endpoint with no session requirement. It opens a connection and runs `SELECT 1`, returning **503** with the driver message if MariaDB is unreachable. It is the fastest way to separate “the API is down” from “the database is down”.

3 · Application architecture

3.1 The four-step journey

`plan-files/demo.md` defines the contract, and the code implements it exactly.

Step	Screen	Behaviour
1	<code>index.html</code>	Role dropdown. An empty selection raises an <code>alert()</code> and blocks navigation.
2	<code>s_login.html</code> / <code>t_login.html</code>	Credential form with a show/hide password toggle. Enter submits.
3	<code>POST /login</code> · <code>POST /teacher-login</code>	Flask verifies against MariaDB, issues a session, returns 200 or 401.
4	<code>student/</code> · <code>teacher/</code>	Dashboard guarded client-side, re-validated server-side on every call.

3.2 Request lifecycle

3.3 Connection and transaction discipline

`server.py` defines one context manager that owns the connection lifecycle:

Three properties follow, and all three matter to the database:

1. **One connection per request, never pooled, never shared.** The connection is opened inside the `with` block and closed in `finally`. There is no long-lived session and no transaction held open between requests, so the schema never has to be session-stateful.
2. **Commit on success, rollback on any exception, close always.** The `/api/teacher/notices` write is therefore atomic: the `INSERT` either lands and commits, or it does not land at all.
3. **A driver error is caught as `mariadb.Error` and mapped to a 500 with the message**, while a missing row produces a 404. Constraint violations surface as 500s with the server's own text.

Every statement in the file uses a `?` placeholder. **No query concatenates a user-supplied value into SQL text.** That invariant holds for all 11 endpoints, including the one statement that takes a `LIMIT` — `LIMIT %s` with the integer passed as a bound parameter, which the `mariadb` driver accepts.

3.4 CORS

```
_cors = os.getenv('CORS_ORIGINS', '').strip()
if _cors:
    CORS(app, resources={ '/': { 'origins': [o.strip() for o in _cors.split(',') if o.strip()] }})
else:
    CORS(app)
```

The static page is served from port 5000 and the API from 8080, so cross-origin requests are unavoidable in local development. If `CORS_ORIGINS` is unset the application allows every origin, which is acceptable for a local demo and unsafe for anything else. Section 7.4 covers it.

3.5 JSON serialisation bridge

MariaDB returns Python types that `jsonify` cannot encode. A three-case adapter bridges the gap:

This is why `CREDITS` is `DECIMAL(3,1)` and `SEM_MARKS` is `DECIMAL(6,2)` rather than `FLOAT`: exact arithmetic for marks, converted to `float` only at the JSON boundary. `dump(rows)` applies it to every value of every row; single-row handlers apply it inline.

NOTE

A rounding artefact worth knowing. `jsonable` turns `Decimal` into `float`, so `SEM_MARKS` of `29.40` is serialised as `29.4`. The screens render it with `scored 29.4 / 30`, while the `percentage` column — computed in SQL as `ROUND(..., 2)` — shows `98.00%`. The stored value keeps its scale; only the transit loses the trailing zero.

4 · The database layer

This is the substance of the project. Thirteen tables, two views, seventeen foreign keys.

4.1 Design objectives

Five goals drove the schema, in priority order.

1. **One authoritative identity per person.** A student is `ROLL`; a teacher is `TID`. Both are `VARCHAR(20)`, both are primary keys, and neither is ever reused. Section 7.1 shows why reuse is a live risk.
2. **Credentials physically separated from profiles.** `STUDENT_LOGIN` and `TEACHER_LOGIN` are two columns wide. A password rotation cannot corrupt a profile row, and a profile edit cannot expose a hash.
3. **Every dashboard figure derived by the database.** No endpoint ships a raw table to JavaScript to be aggregated there. The one exception is documented in Finding 7.5.
4. **Integrity expressed as constraints, not as application code.** A status outside the enumeration cannot reach storage even if the Python regresses.
5. **Every access path index-supported.** This goal is met for identity lookups and partially missed elsewhere; Section 4.8 is honest about which is which.

4.2 The ER model as drawn

`plan-files/ER_MODEL_CONTINUE.drawio` is the design source. It contains one page, 99 cells, **8 entity rectangles, 25 attribute ellipses and 31 connectors**. Because every shape carries an empty `value`, each label is a separate overlaid text cell — so a reader must pair shapes with captions by position. Underlining marks a key attribute.



Figure 2 — ER_MODEL_CONTINUE.drawio, rendered from source

Decoded inventory

Entity (as labelled)	Attributes in the sheet	Underlined
STUDENT	ROLL , NAME , EMAIL , DEPT-SEM	ROLL
STUDENT LOGIN	ROLL , PASSWORD (for now) , NAME	ROLL
TEACHER LOGIN	TID , PASSWORD(for_now) , NAME	TID
TEACHERS	T.ID , T.NAME , T.EMAIL , T.SUBJECT , T.DEPARTMENT	T.ID
COURSES	COURSE_ID , COURSE_NAME	COURSE_ID
STU_COURSES	S.ROLL , C.ID	both
ATTENDANCE	COURSE_ID , ATTENDANCE_STATUS , S.ROLL	COURSE_ID , S.ROLL
ACADEMICS	S.ROLL , MID_SEM , SEM_MARKS	S.ROLL

Connector	Meaning	Cardinality
STUDENT LOGIN → STUDENT	the login row names the same ROLL	1:1
TEACHER LOGIN → TEACHERS	identical arrangement on the staff side	1:1
STUDENT → ACADEMICS	a student accumulates many academic records	1:N

Connector	Meaning	Cardinality
ATTENDENCE → STUDENT	a student is marked many times	:N
COURSES → STUDENT	many-to-many, materialised by STU_COURSES	M:N
COURSES → STU_COURSES	the junction carries both keys	1:N

The other 25 connectors each join one entity to one of its own attributes.

What the sheet does not contain. No `NOTICE` entity, though the student dashboard has a Notices tile. No `STUDY_MATERIAL` entity, though there is a Study materials tile. No line from `COURSES` to `TEACHERS`, though `COURSES.TID` is the ownership column in the shipped schema. No cardinality notation on any edge — all 31 edges have an empty label. The eight drawn entities became thirteen tables; Section 4.3 accounts for every addition.

4.3 From drawing to schema

The shipped schema keeps the author's names and key intent, but the two original composite keys had to be re-cut. This table is the honest delta.

Drawn	Shipped	Why it changed
STUDENT	STUDENT	<code>DEPT-SEM</code> split into <code>DEPT VARCHAR(100)</code> and <code>SEM TINYINT UNSIGNED</code> . A combined string cannot be filtered or indexed by semester.
STUDENT LOGIN	STUDENT_LOGIN	<code>NAME</code> dropped. The drawn duplicate would have been a genuine update anomaly; the login query joins <code>STUDENT</code> for the name instead.
TEACHERS	TEACHER	<code>T.SUBJECT</code> dropped as redundant with <code>DEPT</code> . <code>DESIGNATION</code> added.
TEACHER LOGIN	TEACHER_LOGIN	<code>NAME</code> dropped, same reason.
COURSES	COURSES	<code>CREDITS</code> , <code>TID</code> (owner) and <code>is_active</code> added.
STU_COURSES	STU_COURSES	<code>status</code> <code>ENUM('enrolled','dropped','completed')</code> added so a drop is a data change, not a delete.
ATTENDENCE	ATTENDENCE	Key re-cut. Drawn as <code>(COURSE_ID, S.ROLL)</code> . Shipped as a surrogate <code>ATTENDENCE_ID</code> primary key plus <code>UNIQUE (COURSE_ID, S.ROLL, CLASS_DATE)</code> .
ACADEMICS	ACADEMICS	Key re-cut and split. Drawn as <code>S.ROLL</code> keyed with <code>MID_SEM</code> and <code>SEM_MARKS</code> . Shipped as <code>(ASSESSMENT_ID, S.ROLL)</code> , because marks belong to an <i>assessment</i> , not to a course.
—	ASSESSMENTS	New. The assessment had to become an entity for <code>ACADEMICS</code> to have a sound key.
—	SESSIONS	New. Login state must be somewhere the server can validate.

Drawn	Shipped	Why it changed
—	STUDY_MATERIALS	New. Backs the Study materials tile.
—	STUDY_PROGRESS	New. Junction of students × materials, with a completion state.
—	NOTICES	New. Backs the Notices tile.

Why ATTENDANCE needed a date column

The drawn key (COURSE_ID, S.ROLL) permits exactly **one attendance row per student per course, forever**. It can hold a running percentage but it cannot store history, and it cannot distinguish two classes in one week. The minimum fix is to add the date to the key. The shipped schema does that, and takes the extra step of promoting a surrogate key:

```
ATTENDANCE_ID INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, -- clustered key
UNIQUE KEY uq_attendance (COURSE_ID, `S.ROLL`, CLASS_DATE) -- the real key
```

The UNIQUE constraint is what the business rule actually needs, and it is what guarantees a student cannot be marked twice for one class. The surrogate ATTENDANCE_ID exists so the table has a stable single-column handle and so the natural key can evolve without a rewrite. This is the standard resolution to the “natural key too wide, surrogate key too opaque” tension, and both keys are present. Section 4.7 explains why the surrogate key is never used as an identifier in the API.

Why ACADEMICS needed an assessment entity

As drawn, ACADEMICS keyed on S.ROLL alone means **one academic row per student across all courses combined** — the table cannot represent a student who took five courses. The obvious repair is to add COURSE_ID to the key. The shipped schema instead introduces ASSESSMENTS, keyed on ASSESSMENT_ID, and keys ACADEMICS on (ASSESSMENT_ID, S.ROLL).

That is a strictly better decomposition, and it is worth spelling out why. A course has several assessments — the seed data defines CT-1, CT-2 and Final Exam per course. If ACADEMICS were keyed on (S.ROLL, COURSE_ID) it could still only hold one mark per course, so MAX_MARKS and SEM_MARKS would have to be split into mid_sem and sem_marks columns, exactly as the drawing implies with its MID_SEM / SEM_MARKS pair. That design hard-codes the assumption that every course has exactly two assessments. Promoting the assessment to an entity removes the assumption: a fourth assessment needs a row in ASSESSMENTS, not a schema change.

The `MAX_MARKS` and `WEIGHT` columns also move out of `ACADEMICS` and into `ASSESSMENTS`, which is where they belong — they are properties of the assessment, identical for every student who sat it. That removes a transitive dependency; Section 4.7 shows the working.

4.4 The schema as built

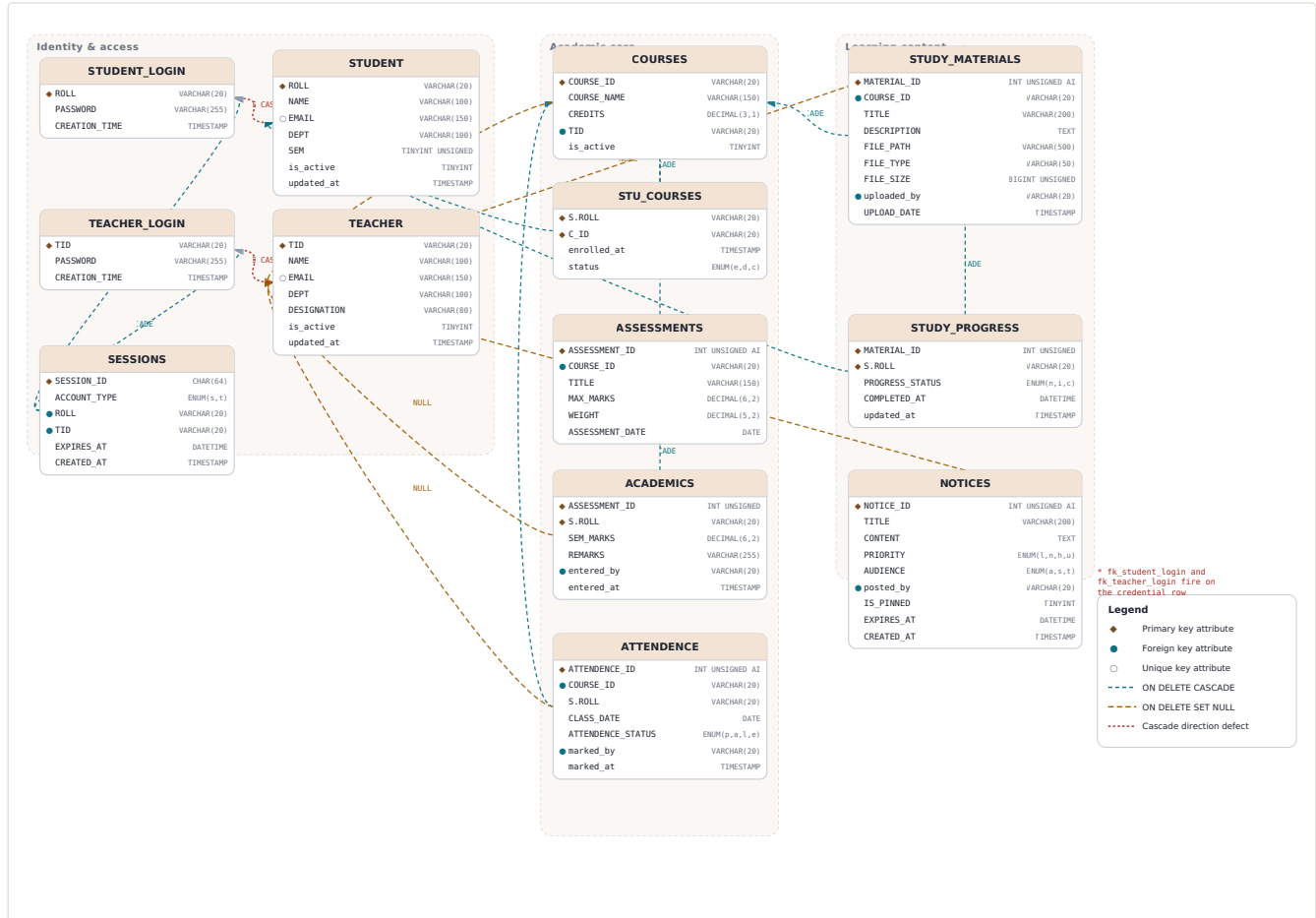


Figure 3 — The as-built schema, generated from the decoded metadata

Three clusters: identity and access on the left, the academic core in the middle, learning content on the right. Dashed teal edges cascade, dashed amber edges null out the reference, and a red note marks the one place the cascade direction is wrong.

4.5 Complete data dictionary

Every column below was read back from `information_schema.COLUMNS` after loading the DDL, so types and nullability reflect what the server actually built rather than what the source file appears to say.

Identity and access

STUDENT_LOGIN — credentials only. 50 rows.

Column	Type	Null	Default	Notes
<code>ROLL</code>	<code>varchar(20)</code>	no	—	PK. Shares the identifier with <code>STUDENT</code> .

Column	Type	Null	Default	Notes
PASSWORD	varchar(255)	no	—	255 chars is sized for a PBKDF2 digest, not a plaintext password.
CREATION_TIME	timestamp	no	current_timestamp()	When the account was created.

STUDENT — public profile. 50 rows.

Column	Type	Null	Default	Notes
ROLL	varchar(20)	no	—	PK , and FK → STUDENT_LOGIN .
NAME	varchar(100)	no	—	Indexed idx_student_name .
EMAIL	varchar(150)	no	—	Unique — two people cannot share an address.
DEPT	varchar(100)	no	—	e.g. Computer Science .
SEM	tinyint unsigned	yes	NULL	Nullable: a newly admitted student may have no semester yet.
is_active	tinyint(1)	no	1	Soft-delete flag; every login query filters on it.
updated_at	timestamp	no	current_timestamp()	ON UPDATE — automatic.

TEACHER_LOGIN / **TEACHER** — structurally identical to the student pair, with **TID** for **ROLL** and an extra DESIGNATION VARCHAR(80) NULL .

SESSIONS — server-side login state. Empty until the first login.

Column	Type	Null	Notes
SESSION_ID	char(64)	no	PK . 64 hex characters, matching secrets.token_hex(32) exactly.
ACCOUNT_TYPE	enum('student','teacher')	no	Required by the @require_session decorator.
ROLL	varchar(20)	yes	FK → STUDENT_LOGIN . NULL for teacher sessions.
TID	varchar(20)	yes	FK → TEACHER_LOGIN . NULL for student sessions.
EXPIRES_AT	datetime	no	Compared with EXPIRES_AT > NOW() .
CREATED_AT	timestamp	no	Audit trail.

The polymorphic-session trick. `SESSIONS` has two nullable owner columns and a discriminator, which is a genuine modelling smell — a discriminator column plus two half-populated foreign keys invites invalid states. `chk_session_owner` closes that hole:

```
CONSTRAINT chk_session_owner CHECK (
  (ACCOUNT_TYPE = 'student' AND ROLL IS NOT NULL AND TID IS NULL) OR
  (ACCOUNT_TYPE = 'teacher' AND TID IS NOT NULL AND ROLL IS NULL))
```

Verified: inserting a `student` row that also carries a `TID` raises `ERROR 4025 ... CONSTRAINT chk_session_owner failed`. The alternative — two separate session tables — would have doubled the endpoint count for no gain.

Academic core

`COURSES` — 5 rows. `COURSE_ID` `VARCHAR(20)` PK, `COURSE_NAME` `VARCHAR(150)`, `CREDITS` `DECIMAL(3,1)` DEFAULT 0.0, `TID` `VARCHAR(20)` NULL (FK, ON DELETE SET NULL), `is_active` `TINYINT(1)` DEFAULT 1.

`STU_COURSES` — junction, 250 rows.

Column	Type	Notes
<code>S.ROLL</code>	<code>varchar(20)</code>	PK part 1. FK → <code>STUDENT</code> , cascade.
<code>C_ID</code>	<code>varchar(20)</code>	PK part 2. FK → <code>COURSES</code> , cascade.
<code>enrolled_at</code>	<code>timestamp</code>	
<code>status</code>	<code>enum('enrolled','dropped','completed')</code>	Seeded as 239 / 3 / 8.

The composite primary key is the textbook resolution of the M:N drawn in Section 4.2. Because `(S.ROLL, C_ID)` is the key, a student cannot enrol twice in one course — the database rejects it, with no `UNIQUE` index needed.

`ASSESSMENTS` — 15 rows (5 courses × 3 assessments).

Column	Type	Notes
<code>ASSESSMENT_ID</code>	<code>int unsigned auto_increment</code>	PK.
<code>COURSE_ID</code>	<code>varchar(20)</code>	FK → <code>COURSES</code> , cascade. Indexed <code>idx_assessments_course</code> .
<code>TITLE</code>	<code>varchar(150)</code>	CT-1, CT-2, Final Exam.
<code>MAX_MARKS</code>	<code>decimal(6,2)</code>	30.00 / 30.00 / 40.00 in the seed.
<code>WEIGHT</code>	<code>decimal(5,2)</code>	20 / 20 / 60 — the syllabus weighting.
<code>ASSESSMENT_DATE</code>	<code>date</code>	

`ACADEMICS` — 750 rows. The fact table of the whole system.

Column	Type	Notes
<code>ASSESSMENT_ID</code>	<code>int unsigned</code>	PK part 1. FK → <code>ASSESSMENTS</code> , cascade.
<code>`S.ROLL`</code>	<code>varchar(20)</code>	PK part 2. No FK — see Section 7.2.
<code>SEM_MARKS</code>	<code>decimal(6,2)</code>	CHECK (<code>SEM_MARKS</code> >= 0).
<code>REMARKS</code>	<code>varchar(255)</code>	Pass / Fail, seeded from a ratio.

Column	Type	Notes
entered_by	varchar(20)	FK → TEACHER , SET NULL . Attribution.
entered_at	timestamp	

Why the backticks. The column is literally named `S.ROLL`, dots included. Every statement touching it is backticked — 14 occurrences across `server.py`, both views and the seed data. Renaming it to `S_ROLL` or `STUDENT_ROLL` would be a readability win; it is not done here because the DDL file is the authoritative artefact and the rename is a coordinated change across four files. Section 7.9 records it as optional cleanup.

ATTENDENCE — 3,000 rows.

Column	Type	Notes
ATTENDENCE_ID	int unsigned auto_increment	PK. Surrogate; never exposed by the API.
COURSE_ID	varchar(20)	FK → COURSES ; part of uq_attendance .
`S.ROLL`	varchar(20)	Part of uq_attendance . No FK — see Section 7.2.
CLASS_DATE	date	Part of uq_attendance .
ATTENDENCE_STATUS	enum('present','absent','late','excused')	Four values, seeded 2,086 / 311 / 291 / 312.
marked_by	varchar(20)	FK → TEACHER , SET NULL .
marked_at	timestamp	

The column name is a misspelling of “attendance”, inherited from the drawing and used consistently across the DDL, the API, the hash routes (`#attendance`) and the screenshots. Correcting it would touch six files for no functional gain, so it is documented rather than changed.

Learning content

STUDY_MATERIALS — 100 rows. `MATERIAL_ID INT UNSIGNED AUTO_INCREMENT PK; COURSE_ID VARCHAR(20) NULL FK cascade; TITLE VARCHAR(200); DESCRIPTION TEXT; FILE_PATH VARCHAR(500); FILE_TYPE VARCHAR(50); FILE_SIZE BIGINT UNSIGNED DEFAULT 0; uploaded_by VARCHAR(20) NULL FK set-null; UPLOAD_DATE TIMESTAMP .`

`FILE_SIZE` is `BIGINT UNSIGNED` rather than `INT`: a 2 GB video lecture would overflow a signed 32-bit column, and the column is also what the front end converts to a human-readable size.

STUDY_PROGRESS — 5,000 rows, the largest table. Junction of students × materials with a completion state.

Column	Type	Notes
MATERIAL_ID	int unsigned	PK part 1. FK → STUDY_MATERIALS , cascade.
`S.ROLL`	varchar(20)	PK part 2. FK → STUDENT , cascade.
PROGRESS_STATUS	enum('not_started','in_progress','completed')	Seeded 2,722 / 1,273 / 1,005.
COMPLETED_AT	datetime	Stamped only for completed rows.
updated_at	timestamp	ON UPDATE .

`COMPLETED_AT` is nullable while `PROGRESS_STATUS` is not. The application convention — visible in the seed's follow-up `UPDATE` — is that `COMPLETED_AT` is written **only** when the status is `completed`, so a non-null `COMPLETED_AT` implies completion. This is a convention, not a constraint: a `CHECK` such as `(PROGRESS_STATUS = 'completed') = (COMPLETED_AT IS NOT NULL)` would enforce it, and is listed in Section 9.

NOTICES — 20 rows.

Column	Type	Notes
<code>NOTICE_ID</code>	<code>int unsigned auto_increment</code>	PK.
<code>TITLE</code>	<code>varchar(200)</code>	The API rejects anything longer.
<code>CONTENT</code>	<code>text</code>	
<code>PRIORITY</code>	<code>enum('low','normal','high','urgent')</code>	Seeded 4 / 10 / 3 / 3.
<code>AUDIENCE</code>	<code>enum('all','students','teachers')</code>	Seeded 8 / 9 / 3.
<code>posted_by</code>	<code>varchar(20)</code>	FK → <code>TEACHER</code> , <code>SET NULL</code> .
<code>IS_PINNED</code>	<code>tinyint(1)</code>	First sort key on the feed.
<code>EXPIRES_AT</code>	<code>datetime</code>	Nullable — <code>NULL</code> means “never expires”.
<code>CREATED_AT</code>	<code>timestamp</code>	

`AUDIENCE` is what makes one table serve two audiences. The student feed filters `IN ('all','students')`; a teacher-only notice never reaches a student. The teacher feed instead filters `posted_by = <own TID>`, so a teacher sees their own posts regardless of audience.

4.6 Keys, constraints and referential actions

4.6.1 The constraint inventory

Every constraint below was verified by inserting a deliberately invalid row and recording the error the server raised. Nothing here is inferred from the source file.

#	Constraint	Type	Table	Deliberate violation	Result
1	<code>PRIMARY KEY</code>	PK	all 13	duplicate <code>STUDENT.ROLL</code>	1062 Duplicate entry 'CS001' for key 'PRIMARY'
2	<code>PRIMARY KEY</code>	composite PK	<code>ACADEMICS</code>	duplicate <code>(1,'CS001')</code>	1062 Duplicate entry '1-CS001' for key 'PRIMARY'
3	<code>chk_marks</code>	CHECK	<code>ACADEMICS</code>	<code>SEM_MARKS = -5</code>	4025 CONSTRAINT <code>chk_marks</code> failed
4	<code>chk_session_owner</code>	CHECK	<code>SESSIONS</code>	<code>student</code> row carrying a <code>TID</code>	4025 CONSTRAINT <code>chk_session_owner</code> failed
5	<code>ATTENDENCE_STATUS</code>	ENUM	<code>ATTENDENCE</code>	'maybe'	1265 Data truncated for column 'ATTENDENCE_STATUS'
6	<code>PRIORITY</code>	ENUM	<code>NOTICES</code>	'critical'	1265 Data truncated for column 'PRIORITY'

#	Constraint	Type	Table	Deliberate violation	Result
7	uq_attendence	UNIQUE	ATTENDENCE	repeat of (C001,CS001,2026-08-03)	1062 Duplicate entry 'C001-CS001-2026-08-03' for key 'uq_attendence'
8	EMAIL	UNIQUE	STUDENT	student01@uni.edu reused	1062 Duplicate entry 'student01@uni.edu' for key 'EMAIL'
9	CLASS_DATE	NOT NULL	ATTENDENCE	CLASS_DATE = NULL	1048 Column 'CLASS_DATE' cannot be null
10	NAME	NOT NULL	STUDENT	NAME = NULL	1048 Column 'NAME' cannot be null
11	fk_attendence_course	FK	ATTENDENCE	unknown COURSE_ID	1452 Cannot add or update a child row: ... fk_attendence_course
12	fk_student_login	FK	STUDENT	ROLL with no login row	1452 ... fk_student_login FOREIGN KEY (ROLL) REFERENCES STUDENT_LOGIN (ROLL)

Test 12 is instructive. Because the constraint is declared **on STUDENT**, the rule enforced is “a profile cannot exist without a credential”, not the reverse. That direction has consequences, and Section 7.1 follows them.

4.6.2 Foreign keys and referential actions

All 17, read from `information_schema.REFERENTIAL_CONSTRAINTS`. `UPDATE_RULE` is `RESTRICT` on every one — an identifier is a primary key and never needs to change, so cascading a rename would be wrong.

Child table	Constraint	Column(s)	→ Parent	ON DELETE
STUDENT	fk_student_login	ROLL	STUDENT_LOGIN	CASCADE △
TEACHER	fk_teacher_login	TID	TEACHER_LOGIN	CASCADE △
SESSIONS	fk_session_student	ROLL	STUDENT_LOGIN	CASCADE
SESSIONS	fk_session_teacher	TID	TEACHER_LOGIN	CASCADE
STU_COURSES	fk_stucourses_student	`S.ROLL`	STUDENT	CASCADE
STU_COURSES	fk_stucourses_course	C_ID	COURSES	CASCADE
STUDY_PROGRESS	fk_studyprog_student	`S.ROLL`	STUDENT	CASCADE
STUDY_PROGRESS	fk_studyprog_material	MATERIAL_ID	STUDY_MATERIALS	CASCADE
COURSES	fk_course_teacher	TID	TEACHER	SET NULL
ASSESSMENTS	fk_assessments_course	COURSE_ID	COURSES	CASCADE
ACADEMICS	fk_academics_assessment	ASSESSMENT_ID	ASSESSMENTS	CASCADE
ACADEMICS	fk_academics_teacher	entered_by	TEACHER	SET NULL
ATTENDENCE	fk_attendence_course	COURSE_ID	COURSES	CASCADE
ATTENDENCE	fk_attendence_teacher	marked_by	TEACHER	SET NULL

Child table	Constraint	Column(s)	→ Parent	ON DELETE
STUDY_MATERIALS	fk_materials_course	COURSE_ID	COURSES	CASCADE
STUDY_MATERIALS	fk_materials_teacher	uploaded_by	TEACHER	SET NULL
NOTICES	fk_notices_teacher	posted_by	TEACHER	SET NULL

FINDING

△ The two login constraints are declared **on the profile table**, referencing the credential table. **ON DELETE CASCADE** therefore means “delete the login row and the profile goes with it” — not the other way round. Deleting a student profile orphans their credential row instead. This is Finding 7.1 and it is the most consequential defect in the schema.

Missing foreign keys. `ACADEMICS.\ S.ROLL`` and `ATTENDENCE.\ S.ROLL`` have no foreign key to `STUDENT`, because `ACADEMICS` already has a composite key ending in `S.ROLL` and `ATTENDENCE` already has `uq_attendence`` in that position, and MySQL/MariaDB will not reuse an existing index as a foreign key target. This is a real referential-integrity hole — Finding 7.2.

4.6.3 Deletion paths, verified by execution

Rather than reason about the cascade graph, the two deletions that matter were run against the loaded schema and the residue counted.

DELETE FROM STUDENT WHERE ROLL = 'CS050';

Table checked	Rows for CS050 afterwards	Interpretation
STUDENT	0	the target, deleted
STUDENT_LOGIN	1	orphaned — the FK points the other way
STU_COURSES	0	cascaded correctly
STUDY_PROGRESS	0	cascaded correctly
ATTENDENCE	60	orphaned — no FK exists
ACADEMICS	15	orphaned — no FK exists

DELETE FROM TEACHER WHERE TID = 'T001';

Table checked	Result	Interpretation
TEACHER	0	the target, deleted
TEACHER_LOGIN	0	cascaded — deleting the credential removed the profile
COURSES	1 row survives, now <code>TID IS NULL</code>	ownerless but intact — correct
ATTENDENCE.marked_by	600 → <code>NULL</code>	history preserved, attribution dropped
ACADEMICS.entered_by	150 → <code>NULL</code>	history preserved
STUDY_MATERIALS.uploaded_by	20 → <code>NULL</code>	history preserved
NOTICES.posted_by	4 → <code>NULL</code>	history preserved

The teacher path is exactly right, and it is the better of the two by a wide margin: an exiting teacher's 774 historical records survive as unattributed facts, while their login vanishes. A **SET NULL** on a nullable attribution column is the textbook answer for “keep the history, drop the attribution”.

4.7 Functional dependencies and normalisation

Each relation is tested against the dependencies its declared keys imply. The target is 3NF, with the fact tables pushed to BCNF where that is free.

Relation	Candidate keys	Dependencies	Form	Note
STUDENT_LOGIN	ROLL	ROLL → PASSWORD, CREATION_TIME	BCNF	Nothing but the key.
TEACHER_LOGIN	TID	TID → PASSWORD, CREATION_TIME	BCNF	Same shape.
STUDENT	ROLL, EMAIL	ROLL → NAME, DEPT, SEM, is_active; EMAIL → ROLL	BCNF	Both keys are unique; every attribute depends on the whole.
TEACHER	TID, EMAIL	TID → NAME, DEPT, DESIGNATION, is_active	BCNF	
COURSES	COURSE_ID	COURSE_ID → COURSE_NAME, CREDITS, is_active; TID → ... (no)	BCNF	CREDITS depends on the course, not the teacher.
STU_COURSES	(S.ROLL, C_ID)	(S.ROLL, C_ID) → enrolled_at, status	BCNF	Full composite-key dependency; nothing on a proper subset.
ASSESSMENTS	ASSESSMENT_ID	ASSESSMENT_ID → COURSE_ID, TITLE, MAX_MARKS, WEIGHT, ASSESSMENT_DATE	BCNF	
ACADEMICS	(ASSESSMENT_ID, S.ROLL)	(ASSESSMENT_ID, S.ROLL) → SEM_MARKS, REMARKS, entered_by, entered_at	BCNF	The key re-cut that removed the transitive dependency — see below.
ATTENDANCE	ATTENDANCE_ID, (COURSE_ID, S.ROLL, CLASS_DATE)	(COURSE_ID, S.ROLL, CLASS_DATE) → ATTENDANCE_STATUS, marked_by, marked_at	BCNF	Two candidate keys; both determinants are superkeys.
STUDY_MATERIALS	MATERIAL_ID	MATERIAL_ID → COURSE_ID, TITLE, ..., UPLOAD_DATE	BCNF	
STUDY_PROGRESS	(MATERIAL_ID, S.ROLL)	(..., S.ROLL) → PROGRESS_STATUS, COMPLETED_AT, updated_at	BCNF	
NOTICES	NOTICE_ID	NOTICE_ID → TITLE, CONTENT, PRIORITY, AUDIENCE, IS_PINNED, EXPIRES_AT	BCNF	

Relation	Candidate keys	Dependencies	Form	Note
SESSIONS	SESSION_ID	SESSION_ID → ACCOUNT_TYPE, ROLL, TID, EXPIRES_AT	BCNF	chk_session_owner supplies the missing dependency.

4.7.1 The anomaly the assessment promotion removed

The drawn design would have been:

```

ACADEMICS (S.ROLL, COURSE_ID) → MID_SEM, SEM_MARKS
      ↑                               ↓
    (partial)                     MAX_MARKS lives here too

```

`MAX_MARKS` depends on `COURSE_ID`, and `COURSE_ID` is only part of the key. That is a transitive dependency — a 3NF violation — and it produces a real update anomaly: changing a course's `MAX_MARKS` would require rewriting every student's row.

The shipped schema places `MAX_MARKS` and `WEIGHT` on `ASSESSMENTS`, keyed by `ASSESSMENT_ID`. Now `MAX_MARKS → ...` is trivial on `ASSESSMENTS`, and `(ASSESSMENT_ID, S.ROLL) → SEM_MARKS` is a full composite-key dependency on `ACADEMICS`. No transitive dependency anywhere. **This one design decision moved the schema from 2NF to BCNF.**

4.7.2 Anomalies avoided

Operation	On an unnormalised design	On this schema
Record a new class day	Overwrite yesterday's status — history is unrecoverable	Insert a new row; <code>CLASS_DATE</code> is part of <code>uq_attendance</code>
Add a fourth assessment	New columns, or a <code>mid4_sem</code> migration	One row in <code>ASSESSMENTS</code>
Change <code>MAX_MARKS</code> for a course	Rewrite every student's marks row	One row in <code>ASSESSMENTS</code>
Delete a course	Hunt down and remove attendance, marks, materials by hand	<code>CASCADE</code> removes all four child sets
A student drops a course	Delete the junction row, losing the drop date	Set <code>status = 'dropped'</code> ; <code>enrolled_at</code> survives
Store a teacher's department	Repeated in every course row	Lives once in <code>TEACHER.DEPT</code>

4.7.3 The one remaining redundancy — and it is deliberate

`COURSES.CREDITS` and `ASSESSMENTS.MAX_MARKS` are both derivable sums in a richer design. They are kept as stored values because the alternative is an aggregate recomputed on every dashboard load, and because

“4 credits” is a *catalogue* fact about the course while a student’s mark is a *fact about that student’s performance*. Storing them is correct denormalisation, not an oversight.

4.8 Indexing strategy

Read back from `information_schema.STATISTICS`. 13 clustered primary keys plus 19 secondary or unique indexes.

4.8.1 What exists, and why

Table	Index	Columns	Justification
STUDENT_LOGIN	PRIMARY	ROLL	The login probe. <code>type: const</code> in <code>EXPLAIN</code> .
STUDENT	PRIMARY	ROLL	
STUDENT	EMAIL	EMAIL	Enforced by <code>UNIQUE</code> ; the index is free.
STUDENT	idx_student_name	NAME	Name search. Not used by any endpoint.
STUDENT_LOGIN →	—	—	
TEACHER, TEACHER_LOGIN	PRIMARY, EMAIL, idx_teacher_name		Mirror of the student side.
SESSIONS	PRIMARY	SESSION_ID	The session probe on every authenticated call.
SESSIONS	idx_sessions_roll	ROLL	Revoke-all-sessions for one student.
SESSIONS	idx_sessions_tid	TID	Revoke-all-sessions for one teacher.
SESSIONS	idx_sessions_exp	EXPIRES_AT	Supports <code>EXPIRES_AT > NOW()</code> and a purge job.
COURSES	PRIMARY	COURSE_ID	
STU_COURSES	PRIMARY	(S.ROLL, C_ID)	Leading column matches the per-student query.
STU_COURSES	idx_stucourses_course	C_ID	Per-course enrolment counts.
ASSESSMENTS	idx_assessments_course	COURSE_ID	FK lookup and the per-course assessment list.
ACADEMICS	PRIMARY	(ASSESSMENT_ID, S.ROLL)	△ Leads with the <i>wrong</i> column for the hot query — Section 7.4.
ATTENDANCE	PRIMARY	ATTENDANCE_ID	Surrogate.
ATTENDANCE	uq_attendance	(COURSE_ID, S.ROLL, CLASS_DATE)	The natural key; also serves the per-course-per-day summary.
ATTENDANCE	idx_att_roll_date	(S.ROLL, CLASS_DATE)	Serves the per-student 30-day window.
ATTENDANCE	idx_att_course_date	(COURSE_ID, CLASS_DATE)	Serves the per-course, per-day teacher view.

Table	Index	Columns	Justification
STUDY_MATERIALS	idx_materials_course	COURSE_ID	FK support.
STUDY_PROGRESS	PRIMARY	(MATERIAL_ID, S.ROLL)	Grain is material-first.
STUDY_PROGRESS	idx_studyprog_roll	(S.ROLL , PROGRESS_STATUS)	Progress-by-student rollup.
NOTICES	idx_notices_feed	(IS_PINNED, CREATED_AT)	Matches ORDER BY IS_PINNED DESC, CREATED_AT DESC exactly — a textbook covering index.

The `ATTENDANCE` triple is the best piece of index design in the schema. One table, three access patterns, three indexes that each match a column order:

Query	Index chosen	EXPLAIN type
per-student summary	idx_att_roll_date	ref
per-course-per-day	uq_attendance	ref, Using index condition
per-student-date window	idx_att_roll_date	ref

4.8.2 Indexes the endpoints do not need yet

`idx_student_name`, `idx_teacher_name`, `idx_sessions_roll`, `idx_sessions_tid` and `idx_sessions_exp` are currently unused — no endpoint searches by name or purges sessions. They are cheap insurance: single-column, on small tables, and each backs an operation an admin will eventually want. They are listed here so the count in the summary is not mistaken for 19 queries that need them.

4.9 Views

Two views, both created with `DROP VIEW IF EXISTS` + `CREATE VIEW` so `mariadb_mysql.sql` stays replayable.

4.9.1 v_student_dashboard

Two decisions are worth defending.

Correlated subqueries rather than joins plus `GROUP BY`. Each figure is an independent aggregate. Five `LEFT JOIN` s with `COUNT(DISTINCT ...)` would multiply the intermediate rows and need five `DISTINCT` s to undo the damage; five correlated subqueries each run once for the single student row. `EXPLAIN` confirms the outer query resolves in one row:

```
id: 1 select_type: PRIMARY table: s type: const
key: PRIMARY ref: const rows: 1
id: 7 select_type: SUBQUERY table: sc type: ref key: PRIMARY rows: 5
id: 7 select_type: SUBQUERY table: m type: ALL key: NULL rows: 100
```

`WHERE is_active = 1` inside the view. This is the row-level security of a small system: a deactivated account returns no dashboard row, so `/api/student/dashboard` answers **404** rather than serving a blank dashboard. The same clause is in `v_teacher_dashboard`.

4.9.2 `v_teacher_dashboard`

Verified output for `T001`:

TID	NAME	DEPT	sections_teaching	total_students	materials_uploaded	notices_posted
T001	Aarav Sharma	Computer Science	1	44	20	4

`total_students` is 44 rather than 50 because it counts only `status = 'enrolled'` — 50 minus 5 completed (`CS001` – `CS005` in `C001`) minus 1 dropped (`CS037` in `C001`). That is the `status` column doing real work.

4.9.3 A view is not a security boundary

`SQL SECURITY DEFINER` is the default, and the stored definition records `DEFINER='correct'@'localhost'`. Neither view exposes `PASSWORD`, and both filter on `is_active`. But a view is a convenience, not a permission: anyone with `SELECT` on the base tables can bypass it. Section 7.4 covers what would be needed to make it one.

4.10 The seed data

`seed_data.sql` is 402 lines and inserts 9,265 rows. It is a **deterministic generator**, not a static dump — which is the more interesting design choice, and the reason two developers can reproduce identical data.

4.10.1 Determinism via `CRC32`

`CRC32` returns a 32-bit unsigned value. `MOD(..., 46)/100` yields a reproducible offset in `[0, 0.45]`, so marks land in `[0.55 × MAX, 1.00 × MAX]` — a plausible distribution that never needs a random seed and never changes between runs. `LEAST()` is the clamp that stops a checksum from producing a mark above the maximum.

The same trick drives attendance and progress:

Generated	Expression	Distribution
<code>ATTENDANCE_STATUS</code>	<code>CASE MOD(CRC32(roll course date)), 10) WHEN 0 THEN 'absent' WHEN 1 THEN 'late' WHEN 2 THEN 'excused' ELSE 'present'</code>	2,086 / 311 / 291 / 312
<code>PROGRESS_STATUS</code>	<code>MOD(CRC32(roll material_id)), 100) < 55 → not_started, < 80 → in_progress, else completed</code>	2,722 / 1,273 / 1,005
<code>SEM_MARKS</code>	<code>0.55 + MOD(CRC32(roll assessment_id)), 46)/100, clamped</code>	pass/fail mixed

Generated	Expression	Distribution
completed_at	DATE_SUB('2026-10-01', INTERVAL MOD(CRC32(...), 45) DAY)	back-dated

4.10.2 CROSS JOIN as a bulk generator

Four tables are generated by cross-join rather than written out: 250 enrolments (50 × 5), 750 marks (15 × 50), 3,000 attendance rows (5 × 12 × 50) and 5,000 progress rows (100 × 50). What would be 9,000 lines of VALUES is four SELECT ... CROSS JOIN statements. The file is reviewable because of this.

4.10.3 The seed data is scenario-complete, and that shows

Three follow-up UPDATE s create the edge cases the screens must survive:

Verified result: **239 enrolled, 3 dropped, 8 completed.** A student who has dropped a course must not see its study materials; a student who has completed one should not see it in their “current” list. That is why /api/student/materials filters sc.status = 'enrolled' while /api/student/academics does not filter at all.

Notices seed all four priorities and all three audiences, four pinned and four with an expiry, so the notice feed's sort order and expiry filter are both exercised.

4.10.4 Two habits worth copying

A helper table, then dropped. SEED_CLASS_DAYS holds the 12 class days, attendance is generated against it, and it is removed at the end:

```
CREATE TABLE IF NOT EXISTS SEED_CLASS_DAYS (CLASS_DATE DATE NOT NULL PRIMARY KEY) ENGINE=InnoDB;
...
DROP TABLE IF EXISTS SEED_CLASS_DAYS;
```

The shipped schema is left with exactly 13 tables, none of them scaffolding.

Verification queries live in the seed file. Three read-only SELECT s at the end check the per-course material rollup, one student's full marksheet, and one student's attendance breakdown. A reviewer can prove the load worked without writing a query.

5 · Every endpoint, and the SQL behind it

Eleven endpoints. Each entry gives the route, the guard, the statement as written in `server.py`, and the execution plan as returned by `EXPLAIN` against the loaded schema.

5.1 Endpoint summary

#	Route	Method	Guard	Statement	Writes
1	<code>/login</code>	POST	none	join <code>STUDENT_LOGIN</code> + <code>STUDENT</code>	<code>SESSIONS</code> , maybe <code>STUDENT_LOGIN</code>
2	<code>/teacher-login</code>	POST	none	join <code>TEACHER_LOGIN</code> + <code>TEACHER</code>	<code>SESSIONS</code> , maybe <code>TEACHER_LOGIN</code>
3	<code>/logout</code>	POST	none	<code>DELETE FROM</code> <code>SESSIONS</code>	✓
4	<code>/api/student/ dashboard</code>	POST	student	view + aggregate	
5	<code>/api/student/ materials</code>	POST	student	3-table join	
6	<code>/api/student/ academics</code>	POST	student	3-table join	
7	<code>/api/student/ attendance</code>	POST	student	2-table join, <code>GROUP BY</code>	
8	<code>/api/student/ notices</code>	POST	student	left join + filters	
9	<code>/api/teacher/ dashboard</code>	POST	teacher	view	
10	<code>/api/teacher/ courses</code>	POST	teacher	<code>LEFT JOIN</code> + <code>GROUP BY</code>	
11	<code>/api/teacher/ academics</code>	POST	teacher	4-table join	
12	<code>/api/teacher/ attendance</code>	POST	teacher	2-table join, <code>GROUP BY</code> , <code>LIMIT</code>	
13	<code>/api/teacher/ notices</code>	GET	teacher	own notices	
14	<code>/api/teacher/ notices</code>	POST	teacher	<code>INSERT</code>	✓
15	<code>/health</code>	GET	none	<code>SELECT 1</code>	

5.2 The session guard

Every authenticated endpoint is wrapped in a decorator that resolves the token to an identity before the view function runs.

Four properties, all deliberate:

1. **Three ways to present the token** — JSON body, `X-Session-Id` header, or neither. The header exists because a `GET` cannot carry a body; the teacher notices feed uses it.
2. **`ACCOUNT_TYPE = ?` in the predicate.** A student token cannot call a teacher endpoint even if it is valid and unexpired. This is the single most important line in the file.
3. **`EXPIRES_AT > NOW()` in the predicate.** Expiry is decided by the database clock, not by the browser and not by Python's clock. There is no timer job and no cleanup required — an expired row simply stops matching.
4. **`ACCOUNT_TYPE = 'student'` is a parameter, never interpolated,** even though the two legal values are known constants. The table itself enforces the invariant with `chk_session_owner`.

`g.owner` then becomes the only value the endpoint needs. Because the client already knows its own identity, **every dashboard read is parameterised by a single value**, which is why the index set in Section 4.8 is small: one primary-key probe or one `ref` seek per screen.

5.3 Authentication

`POST /login`

```
SELECT l.PASSWORD, p.NAME
FROM STUDENT_LOGIN l
JOIN STUDENT p ON p.ROLL = l.ROLL
WHERE l.ROLL = ? AND p.is_active = 1
```

The inner join does three jobs at once: it looks up the credential, it fetches the authoritative name from the profile (so `STUDENT_LOGIN` needs no `NAME` column), and it enforces `is_active = 1` without a second round trip.

`EXPLAIN`: `type: eq_ref` on both tables via `PRIMARY`. One row, always.

Outcome	Status	Body
row absent — unknown roll, or inactive	401	<code>roll number does not exist</code>
row present, hash mismatch	401	<code>wrong password</code>
row present, hash matches	200	<code>{message, roll, name, session_id}</code>

The two 401 messages are deliberately different — one discloses that a roll number is unregistered. Harmless for a demo, and a username-enumeration vector in production. Section 7.3.

`/teacher-login` is the same statement against `TEACHER_LOGIN` and `TEACHER`, keyed on `TID`, and it also accepts `tid` sent as `roll` for convenience.

5.4 Student reads

`/api/student/dashboard` — the tiles

The view read is one row. The average-score tile is a second statement:

Read the arithmetic carefully, because it is doing something non-obvious. It is a *ratio of sums, averaged across courses* — not an average of ratios. For each course it totals marks obtained over marks possible, converts to a percentage, then averages those five percentages unweighted.

This is defensible and arguably the fairer metric: a 40-mark final exam is not counted four times as heavily as a 30-mark class test. It is also the metric a transcript should not use, where a weighted mean via `ASSESSMENTS.WEIGHT` would be correct. `WEIGHT` is seeded (20/20/60) and currently unused by any endpoint — an obvious improvement, listed in Section 9.

EXPLAIN of the outer statement:

```
id: 1  select_type: PRIMARY  table: <derived2>  type: ALL  rows: 5
id: 2  select_type: DERIVED  table: a          type: ALL  rows: 15  ← full scan
id: 2  select_type: DERIVED  table: m          type: eq_ref key: PRIMARY rows: 1
```

`a` is scanned in full and `ACADEMICS` is probed 15 times, because the derived table is materialised before the `WHERE` clause is applied — a known MariaDB behaviour with `GROUP BY` inside a derived table. `ACADEMICS` itself is reached through `eq_ref` on the primary key, so the 750-row table is *not* scanned. The result for `CS001` is `78.04`.

/api/student/materials

The only endpoint that filters on enrolment status, and correctly so: a student who has **dropped** a course must not see its materials. Verified — `CS037` dropped `C001`, so that pairing returns nothing.

EXPLAIN :

```
table: sc  type: ref  key: PRIMARY  rows: 5  Extra: Using where; Using temporary; Using filesort
table: c   type: eq_ref key: PRIMARY  rows: 1
table: m   type: ALL  key: NULL      rows: 100 Extra: Using join buffer (flat, BNL join)
```

`STU_COURSES` narrows to 5 courses via the leading column of its primary key; `STUDY_MATERIALS` is then joined without an index, so the optimiser picks a block nested loop. At 100 rows that is the right call — a 20-row index probe per course would cost more in random I/O. **This plan is correct at the current scale and will stop being correct somewhere around 50,000 materials**, at which point the join order should be forced to `sc → c → m` with `idx_materials_course`. That is the right way to frame it: not a mistake, but a ceiling.

/api/student/academics

The chain `ACADEMICS → ASSESSMENTS → COURSES` is exactly the normalisation from Section 4.3 in use: the marksheet is assembled by walking from the fact table to the assessment to the course name.

`m.SEM_MARKS <= a.MAX_MARKS` is a defensive guard, not a constraint. The schema enforces `SEM_MARKS >= 0` but has no upper bound, because `MAX_MARKS` lives in a *different table* and a `CHECK` cannot cross tables. So an application bug could store a mark above the maximum, and this predicate hides it from the student rather than displaying an impossible 140%. The honest fix is a trigger — and indeed a stronger reason to prefer a weighted design over cross-table `CHECK`.

No enrolment filter here, unlike the materials query. That asymmetry is deliberate in effect: a student keeps access to their marks after completing a course, but not to that course's materials. Verified — `CS001` completed `C001` and still sees all three `C001` assessments.

`/api/student/attendance`

MySQL's `SUM(boolean)` idiom, with `CAST(... AS SIGNED)` because the driver returns a `Decimal` that `jsonable` would turn into `10.0` rather than `10`. The `100.0` multiplier forces decimal division so `ROUND` is meaningful. Grouping by both `a.COURSE_ID` and `c.COURSE_NAME` satisfies `ONLY_FULL_GROUP_BY`.

Verified output for `CS001`:

COURSE_ID	COURSE_NAME	classes_held	present_days	late_days	absent_days	attended	percentage
C002	Discrete Math	12	9	1	0	10	83.33
C004	Pentesting	12	8	1	1	9	75.00
C001	Programming	12	10	1	0	11	91.67
C003	Reverse Engineering	12	8	1	0	9	75.00
C005	Soc	12	8	2	1	10	83.33

`EXPLAIN` confirms `uq_attendance` is chosen with `Using index condition`.

FINDING

The three counts do not sum to `classes_held`. `CS001` holds **9 excused rows** across the 12 days. The `ENUM` has four values; the query enumerates three. An excused day is counted in the denominator `COUNT(*)` — so it lowers the percentage — but is itemised in none of the three columns. The UI prints “11 of 12 classes attended” with present/late/absent summing to 11, and the unexplained twelfth is an excused absence. All 50 students have excused rows. This is Finding 7.6.

`/api/student/notices`

Four details, all correct.

- `LEFT JOIN` on `TEACHER`, so a notice whose author has been deleted still appears, with `posted_by` falling back to the string `'staff'` in JavaScript. This is exactly what `fk_notices_teacher`'s `SET NULL` is for.
- `EXPIRES_AT IS NULL OR EXPIRES_AT > NOW()` — `NULL` means “never expires”, and the two-branch test is required; a plain `EXPIRES_AT > NOW()` would drop every permanent notice.
- `ORDER BY IS_PINNED DESC, CREATED_AT DESC` matches `idx_notices_feed` column for column.
- `LIMIT %s` with a bound parameter, clamped in Python to `max(1, min(limit, 50))` so a client cannot ask for the whole table.

5.5 Teacher reads and the write path

`/api/teacher/courses`

The `status = 'enrolled'` predicate belongs in the `ON` clause, not `WHERE`. In `WHERE` it would convert the outer join to an inner one and silently drop a course with zero enrolments — a teacher with a new section would see nothing. As written, `COUNT(sc.S.ROLL)` returns 0 for that course and the row survives. This is the correct placement and the reason the query works at all.

`EXPLAIN`: `COURSES` is reached with `key: fk_course_teacher ( type: ref , rows: 1 )` — the teacher owns one course. `STU_COURSES` is then joined without an index (`type: ALL , rows: 250 , BNL join`). Correct again: filtering 5 course rows to 1 and then scanning 250 enrolments beats 50 index probes.

`/api/teacher/academics`

The deepest query in the system — four tables:

Ownership is expressed once, as `c.TID = ?`. Everything else is a consequence of normalisation: the teacher reaches their courses through `COURSES`, their assessments through `ASSESSMENTS`, and their students through `ACADEMICS`. **The teacher never filters on `ACADEMICS` directly and never sees another teacher's marks, with no row-level filter in Python.** The access rule is expressed once, in one predicate, at the right place in the graph. This is the single best argument for the normalisation work in Section 4.3.

`/api/teacher/attendance`

`CLASS_DATE` is the grouping key because the screen shows one row per class session. `LIMIT 60` is hard-coded — 12 class days × 5 courses is 60, the exact size of the seed, so the seed exercises the limit precisely. `idx_att_course_date` would serve this grouping directly if the optimiser chose it; on the current data it prefers `COURSES` first and joins per course. Both orders are correct.

`POST /api/teacher/notices` — the only write

```
INSERT INTO NOTICES (TITLE, CONTENT, PRIORITY, AUDIENCE, posted_by)
VALUES (?, ?, ?, ?, ?)
```

`posted_by` is `g.owner` — the teacher resolved from the session token, **not** anything the client sent. A client cannot post a notice as someone else. The validation is a whitelist, and it mirrors the schema:

This validation is technically redundant — the `ENUM` columns would reject a bad value and `VARCHAR(200)` would truncate or reject an over-long title. It is kept because a **400 with a readable message** beats a **500 carrying raw driver text**, and because it costs four lines. The database remains the real authority; the application just fails earlier and better.

On success the client invalidates two caches and refetches:

```
loaded.delete('notices'); await loadPage('notices');
loaded.delete('home');    loadHome();
```

which is why the notice count tile updates without a page reload.

`GET /api/teacher/notices` is the same route with the complementary filter:

```
SELECT n.NOTICE_ID, n.TITLE, n.CONTENT, n.PRIORITY, n.AUDIENCE, n.IS_PINNED, n.CREATED_AT  
FROM NOTICES n WHERE n.posted_by = ? ORDER BY n.CREATED_AT DESC LIMIT 50
```

Student-scoped feeds filter by `AUDIENCE` ; teacher-scoped feeds filter by `posted_by` . Each audience sees its own list from one table.

6 · Security design

6.1 Password hashing

The DDL comment reads `PASSWORD (for now)`, and that note has been honoured: the column is `VARCHAR(255)` — sized for a digest, not a plaintext password — and the seed stores `stud123` in the clear as a starting state.

Stored as `pbkdf2_sha256$200000$<salt-hex>$<digest-hex>`. 16 random bytes of salt per user, 200,000 iterations, SHA-256. Comparison is constant-time:

```
return compare_digest(expected, actual.hex()), False
```

Four design points, all correct.

1. **The iteration count is stored in the string, not hard-coded at verify time.** When the parameter is raised to 600,000, old hashes keep verifying — and `verify_password` reads `rounds` from the stored value.
2. **`verify_password` returns a two-tuple `(ok, is_plain)`.** A stored value without the `pbkdf2_sha256$` prefix is compared directly with `compare_digest` and flagged as plaintext.
3. **Plaintext is migrated transparently on first successful login.** The login handler does, inside the same transaction:

```
if is_plain and ok:
    cur.execute('UPDATE STUDENT_LOGIN SET PASSWORD = ? WHERE ROLL = ?',
                (hash_password(password), roll))
```

Seeded demo accounts upgrade themselves on first use. A correct answer to the password column as a schema type.

4. **No application-level hashing round-trip.** The login query returns `l.PASSWORD` and the verification happens in Python, because the database has no `PBKDF2` function to call. The database never needs to know a plaintext password.

6.2 Server-side sessions

Sessions live in `SESSIONS`, not in a signed cookie and not in Python memory. That decision buys three things:

- **Immediate revocation.** `POST /logout` deletes the row. With a stateless JWT the token would remain valid until expiry.
- **A server-side clock.** `EXPIRES_AT > NOW()` uses MariaDB's `NOW()`, so a client with a wrong system clock gains nothing.
- **Role binding in the key.** `ACCOUNT_TYPE` is part of the lookup, so a token cannot cross the student/teacher boundary.

The token itself is `secrets.token_hex(32)` — 32 bytes from the OS CSPRNG, hex encoded to exactly 64 characters. That is why the column is `CHAR(64)` and not `VARCHAR(255)`: a fixed-width column has no length ambiguity, and the Python constant and the DDL agree by construction.

The 8-hour TTL (`SESSION_TTL_HOURS = 8`) is a reasonable compromise for a demo and too long for production, where it should drop to 30 minutes with a sliding refresh. Section 7.4.

6.3 What the design gets right

Control	Implementation
Passwords never stored in the clear after first login	Transparent PBKDF2 migration, §6.1
Per-user random salt	<code>secrets.token_hex(16)</code>
Constant-time comparison	<code>hmac.compare_digest</code>
Password column sized for a hash	<code>VARCHAR(255)</code>
Parameterised statements	<code>?</code> placeholders on all 15 statements, including <code>LIMIT %s</code>
Session token from a CSPRNG	<code>secrets.token_hex(32)</code> → <code>CHAR(64)</code>
Expiry enforced by the database	<code>EXPIRES_AT > NOW()</code> in the guard predicate
Role enforced server-side	<code>ACCOUNT_TYPE = ?</code> on every lookup
Author identity not client-supplied	<code>posted_by = g.owner</code>
Soft deletion	<code>is_active</code> on <code>STUDENT</code> and <code>TEACHER</code>
Passwords never leave the server	Neither view selects <code>PASSWORD</code>

6.4 What a production deployment would still need

Gap	Impact	Fix
CORS allows all origins by default	Any site can call the API from a victim's browser	Set <code>CORS_ORIGINS</code> ; drop the bare <code>CORS(app)</code> branch
No <code>SECRET_KEY</code> in use	Flask sessions unusable — fine, since sessions are in the database	Keep it that way, or set one and use Flask sessions
8-hour session TTL	Excessive window after compromise	30 minutes + sliding refresh
Differentiated 401 messages	Reveals which roll numbers exist	One message for both failure modes
No failed-login throttling	Unlimited password guessing	Per-IP and per-account backoff
No password complexity rule	<code>stud123</code> satisfies everything	Validate at write time; there is no signup path yet
<code>SQL SECURITY DEFINER</code> views	View access implies base-table access	Grant on views only, or use <code>SQL SECURITY INVOKER</code>
No audit table	No record of who read what	<code>LOGIN_AUDIT(who, role, success, at)</code>
No <code>expired</code> session purge	<code>SESSIONS</code> grows without bound	<code>DELETE FROM SESSIONS WHERE EXPIRES_AT < NOW()</code> on a schedule — <code>idx_sessions_exp</code> is already in place for it

That last row is a nice illustration of the index strategy in Section 4.8.2: the index for the purge job exists before the job does.

7 · Verified findings

Every finding below was reproduced by executing against the loaded schema. Each carries a severity, the reproduction, and a concrete fix. They are ordered by severity.

7.1 HIGH — `ON DELETE CASCADE` on the login keys points the wrong way

Where. `mariadb_mysql.sql`, `fk_student_login` and `fk_teacher_login`.

The defect. The constraint is declared on the *profile*:

```
CREATE TABLE STUDENT (
  ROLL VARCHAR(20) NOT NULL PRIMARY KEY,
  ...
  CONSTRAINT fk_student_login FOREIGN KEY (ROLL) REFERENCES STUDENT_LOGIN(ROLL) ON DELETE CASCADE
);
```

`ON DELETE CASCADE` propagates **from parent to child**. The parent here is `STUDENT_LOGIN`. So the rule that actually holds is “delete the credential and the profile disappears” — not “delete the profile and the credential disappears”.

Reproduction. On a clean seeded database:

```
DELETE FROM STUDENT WHERE ROLL = 'CS050';
```

Check	Result	Expected
<code>STUDENT</code>	0	0
<code>STUDENT_LOGIN</code>	1	0
<code>STU_COURSES</code>	0	0
<code>STUDY_PROGRESS</code>	0	0
<code>ATTENDANCE</code>	60	0
<code>ACADEMICS</code>	15	0

The credential row survives, and 75 rows of marks and attendance survive with no student to own them.

Why it is a security issue, not just untidiness. The orphaned `STUDENT_LOGIN` row is a dormant credential. Reproduced end to end:

```
seeded:      SELECT PASSWORD FROM STUDENT_LOGIN WHERE ROLL='CS050'; → stud123
DELETE FROM STUDENT WHERE ROLL='CS050';
re-register:  INSERT INTO STUDENT (ROLL,NAME,EMAIL,DEPT,SEM)
              VALUES ('CS050','New Person','cs050@uni.edu','Computer Science',1);

then the exact query /login runs:
SELECT l.ROLL, l.PASSWORD, p.NAME FROM STUDENT_LOGIN l
JOIN STUDENT p ON p.ROLL = l.ROLL WHERE l.ROLL='CS050' AND p.is_active=1;

          ROLL    PASSWORD    NAME
          CS050    stud123    New Person
```

A newly registered student authenticates with the **previous holder's** password. The API cannot prevent this — it has no way to know the credential predates the profile.

Fix. Move the constraint to the credential table so `STUDENT_LOGIN` is the child:

Same edit for `TEACHER_LOGIN` / `fk_teacher_login`. The parent-child direction now matches the intent, and `CHECK (ACCOUNT_TYPE ...)` still guarantees a credential is never shared between a student and a teacher.

Interim mitigation, zero DDL changes: never `DELETE` from `STUDENT`. Use `UPDATE STUDENT SET is_active = 0 WHERE ROLL = ?` instead. That is already the intended soft-delete path, every query already honours `is_active`, and it closes the exposure.

7.2 HIGH — `ACADEMICS` and `ATTENDANCE` have no foreign key to `STUDENT`

Where. `mariadb_mysql.sql`.

The defect. Both tables store ``S.ROLL`` with no reference to `STUDENT`. Verified:

```
INSERT INTO ACADEMICS (ASSESSMENT_ID, `S.ROLL`, SEM_MARKS) VALUES (1, 'CS999', 10);
-- ACCEPTED. No error. A roll number that does not exist is now in the marks table.
```

The cause is structural. `ACADEMICS`'s primary key already ends in `S.ROLL`, and `ATTENDANCE`'s `uq_attendance` already contains it in position 2, and MySQL/MariaDB will not reuse an existing index to satisfy a new foreign key. So this is not carelessness — it is a consequence of the composite-key design, which makes the omission easy to overlook.

Impact. 750 marks rows and 3,000 attendance rows have no guaranteed relationship to any student. A typo in a bulk insert produces marks for a nonexistent person, and no query will ever surface them, because every read endpoint filters by a real roll number first.

Fix. Add the foreign keys, which requires indexes that lead with `S.ROLL`:

This also **fixes the index problem in Finding 7.4** as a side effect: the required `S.ROLL`-leading index is exactly the index the dashboard query needs. One fix, two findings.

7.3 MEDIUM — `recent_notices` is a global count, not a per-student one

Where. `v_student_dashboard`.

The defect.

```
(SELECT COUNT(*) FROM NOTICES n
 WHERE n.CREATED_AT >= CURDATE() - INTERVAL 7 DAY
 AND n.AUDIENCE IN ('all','students')) AS recent_notices
```

The subquery references `n` only. It never correlates to `s.ROLL` — so it returns **the same number for every student in the system**. Verified: `CS001` returns `17`, and so does every other roll.

It should be `COUNT(DISTINCT n.NOTICE_ID)` joined through the student's enrolled courses, or accepted as a campus-wide announcement counter. As written it is named `recent_notices`, displayed under a tile labelled “recent”, and implies a personal feed.

Fix. Either rename the column to `campus_notices` and the tile label to “campus”, or correlate it properly. The current behaviour is not harmful — it is mislabelled.

7.4 MEDIUM — `ACADEMICS`’ primary key leads with the wrong column

Where. `mariadb_mysql.sql`, `ACADEMICS PRIMARY KEY (ASSESSMENT_ID, S.ROLL)`.

The defect. In InnoDB the clustered primary key *is* the table’s physical order, and a secondary index can only be probed with a **leading-column** predicate. The hottest marks query filters on `S.ROLL`:

```
WHERE m.`S.ROLL` = ?
```

Because `S.ROLL` is the second column of the primary key, that predicate cannot use it. Verified plan for the student dashboard aggregate:

```
id: 2  select_type: DERIVED  table: a  type: ALL  rows: 15  Extra: Using temporary; Using filesort
id: 2  select_type: DERIVED  table: m  type: eq_ref key: PRIMARY  rows: 1
```

`ASSESSMENTS` is fully scanned, and `ACADEMICS` is probed once per assessment — 15 probes, each matching, rather than one `ref` seek on `S.ROLL`. Every student dashboard load pays it.

Fix. Add the index — and note it is the same one Finding 7.2 needs:

```
ALTER TABLE ACADEMICS ADD KEY idx_academics_roll (`S.ROLL`);
```

Expected result: `type: ref`, `key: idx_academics_roll`, one seek per student.

7.5 MEDIUM — the seed data leaves the 30-day attendance window stale

Where. `seed_data.sql` versus `v_student_dashboard`.

The defect. The dashboard counts attendance within `CLASS_DATE >= CURDATE() - INTERVAL 30 DAY`. The seed generates 12 class days between **2026-08-03** and **2026-09-10**. Once the system clock passes **2026-10-10**, every seeded class day falls outside the window and the tile reads `0%`.

Verified at the time of writing (2026-10-05):

```
today          2026-10-04
attendance window >= 2026-09-04    ← only 2 of 12 class days qualify

SELECT * FROM v_student_dashboard WHERE ROLL='CS001';
  present_last_30:  6
   total_last_30: 10
```

The tile reports **60%** from 10 of the student’s 60 attendance records. The attendance *page* — which has no date window — correctly shows 43 present, 6 late, 2 absent, 9 excused. **The two screens disagree**, and the dashboard is the one that is wrong.

Fix. Make the seed relative to the load date rather than hard-coded:

```
-- instead of literal dates
SET @today = CURRENT_DATE;
INSERT INTO SEED_CLASS_DAYS (CLASS_DATE) VALUES (@today - INTERVAL 11 WEEKDAY DAY);
```

Or accept it and document that the demo’s date window is 2026-08 to 2026-10. A one-line comment in `seed_data.sql` stating the seed’s validity period would prevent the confusion.

7.6 MEDIUM — `excused` attendance is counted but never shown

Where. `/api/student/attendance`.

The defect. `ATTENDANCE_STATUS` is `ENUM('present','absent','late','excused')`. The query sums three of the four values but uses `COUNT(*)` as the denominator. Verified for `CS001`:

Field	Value
<code>classes_held</code>	12
<code>present_days</code>	10
<code>late_days</code>	1
<code>absent_days</code>	0
sum of the three	11
<code>attended</code>	11
<code>percentage</code>	91.67

$10 + 1 + 0 = 11 \neq 12$. The missing day is excused. All 50 students have excused rows, so this affects every account. The UI renders “11 of 12 classes attended — 10 present, 1 late, 0 absent”, which does not add up and gives the student no way to learn that an excused absence was counted against them.

Fix. Add the fourth column and exclude excused from the denominator, which is the usual academic convention:

```
CAST(SUM(a.ATTENDANCE_STATUS = 'excused') AS SIGNED) AS excused_days,
...
ROUND(100.0 * SUM(a.ATTENDANCE_STATUS IN ('present','late'))
/ SUM(a.ATTENDANCE_STATUS <> 'excused'), 2) AS percentage
```

Then render `excused_days` in the `data-sub` line, which already has room.

7.7 LOW — `new_materials` counts all 100 materials as “new this week”

Where. `v_student_dashboard`.

The defect. `STUDY_MATERIALS.UPLOAD_DATE` is `TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP`, and the seed inserts 100 rows with literal dates in `TITLE / FILE_PATH` but no explicit `UPLOAD_DATE`. So every row is stamped with the moment the seed ran, and the tile’s `UPLOAD_DATE >= CURDATE() - INTERVAL 7 DAY` test matches all 100.

Verified: `new_materials: 100`, and `MIN(UPLOAD_DATE) = MAX(UPLOAD_DATE)` — a single timestamp for the entire table.

Fix. Back-date a spread of rows in the seed, the same way the notices and attendance data already are:

```
UPDATE STUDY_MATERIALS SET UPLOAD_DATE = TIMESTAMP(DATE_SUB(CURRENT_DATE, INTERVAL MOD(MATERIAL_ID, 40) DAY))
WHERE MOD(MATERIAL_ID, 3) <> 0; -- leave a third as genuinely new
```

7.8 LOW — `NOTICES.EXPIRES_AT` can silently hide a notice forever

Where. `/api/student/notices`.

The defect. The filter is `(n.EXPIRES_AT IS NULL OR n.EXPIRES_AT > NOW())`. A notice posted with an `EXPIRES_AT` in the past is invisible to every student and gives no feedback to the teacher who posted it — the write endpoint returns `201 notice posted` with no indication the notice will never be seen.

Fix. Cheap warning at write time; a real fix is to reject a past expiry:

```
if data.get('expires_at') and data['expires_at'] < datetime.now().isoformat():
    return jsonify({'message': 'expires_at is in the past'}), 400
```

7.9 LOW — cosmetic and consistency items

Item	Detail	Suggestion
<code>ATTENDENCE</code> misspelling	Inherited from the drawing; consistent across DDL, API, hash routes and screenshots	Leave, or rename in one coordinated commit
<code>`S.ROLL`</code> column name	A dot in an identifier forces backticks on 14 occurrences across four files	<code>STUDENT_ROLL</code> ; rename is mechanical but must be atomic
<code>STUDY_MATERIALS.FILE_PATH</code> unused	The column stores <code>materials/C001/unit-01-notes.pdf</code> but no endpoint returns it — only <code>FILE_TYPE</code> and <code>FILE_SIZE</code>	Either expose a download route or drop the column
<code>notices.posted_by</code> naming	The column name differs only in case from every other identifier, which are <code>UPPER_CASE</code>	<code>POSTED_BY</code> for consistency
<code>ASSESSMENTS.WEIGHT</code> unused	Seeded 20/20/60 but read by no endpoint — see §5.4	Use it for a weighted average, the correct transcript metric
<code>db()</code> per request	A connection is opened and closed per statement; <code>STUDENT_LOGIN</code> holds one statement per request	A pool (<code>mariadb pool()</code> or <code>Flask g</code>) — an optimisation, not a defect
<code>teacher_academics</code> unpaginated	Returns every mark for every student in a teacher's courses — 375 rows for <code>T001</code>	Add a <code>LIMIT</code> like the attendance endpoint has

7.10 Severity summary

Severity	Count	Findings
HIGH	2	7.1 reversed cascade with credential inheritance; 7.2 missing <code>STUDENT</code> foreign keys
MEDIUM	4	7.3 global notice count; 7.4 wrong key column order; 7.5 stale seed window; 7.6 excused attendance

Severity	Count	Findings
LOW	6	7.7, 7.8, plus five items in 7.9

The two HIGH findings are both about the same relationship — *student to credential* — and both are fixed by the same pair of edits: move the two login foreign keys onto the credential tables, and add **S.ROLL** foreign keys plus indexes to **ACADEMICS** and **ATTENDENCE** .

8 · The application, screen by screen

Every figure in this section is a capture from `output-files/`. Each caption names the endpoint whose response populates the screen, so the UI and the schema can be read together.

8.1 Role selection

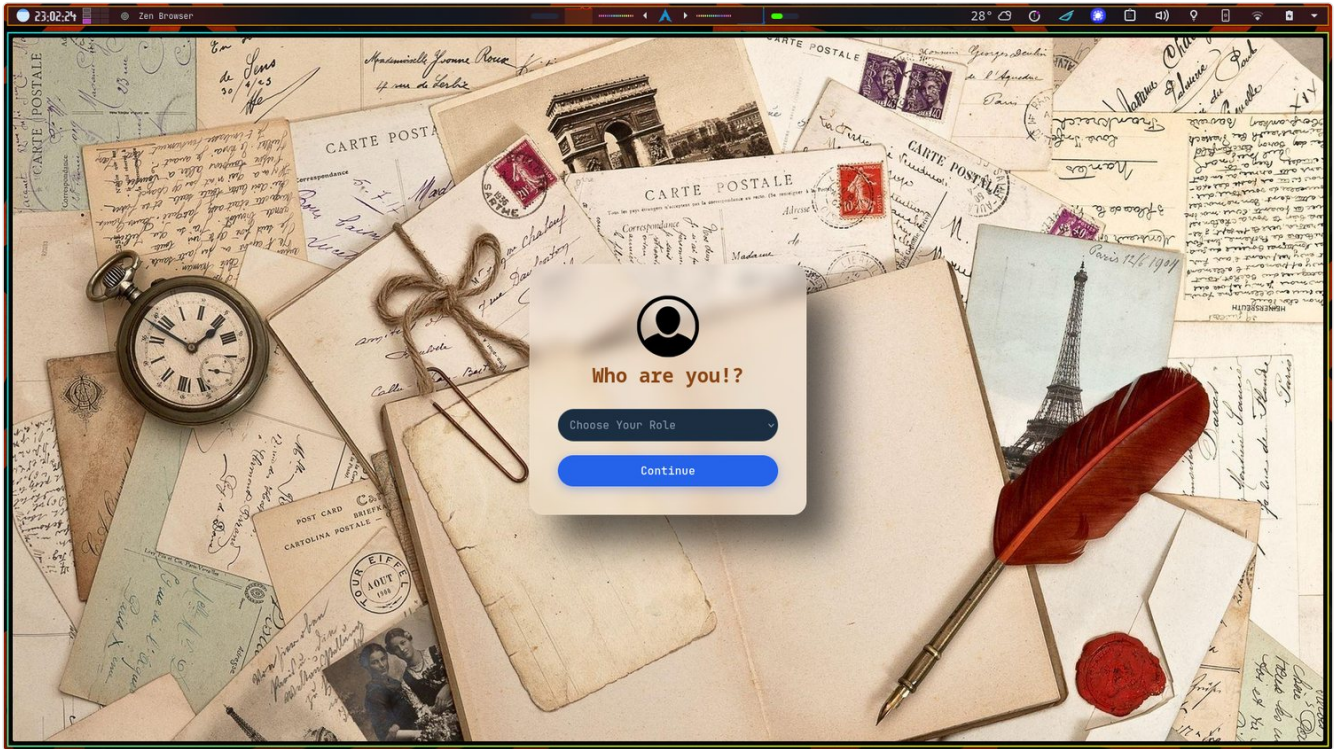


Figure 4 — Role selector. No database involvement.

`index.html` asks one question and reads no data. The guard is three lines of JavaScript — an empty selection raises an `alert()` and returns before navigation. `Enter` in the `<select>` triggers the button, so keyboard-only users are not stranded.

Note what this screen establishes architecturally: **the role is chosen by the user, not discovered by the system.** The next screen hardcodes which login endpoint and which dashboard follow. The

database's `ACCOUNT_TYPE` is therefore never inferred from context — it is asserted by the route the client chose, and re-verified by the `SESSIONS` table on every request.

8.2 Login

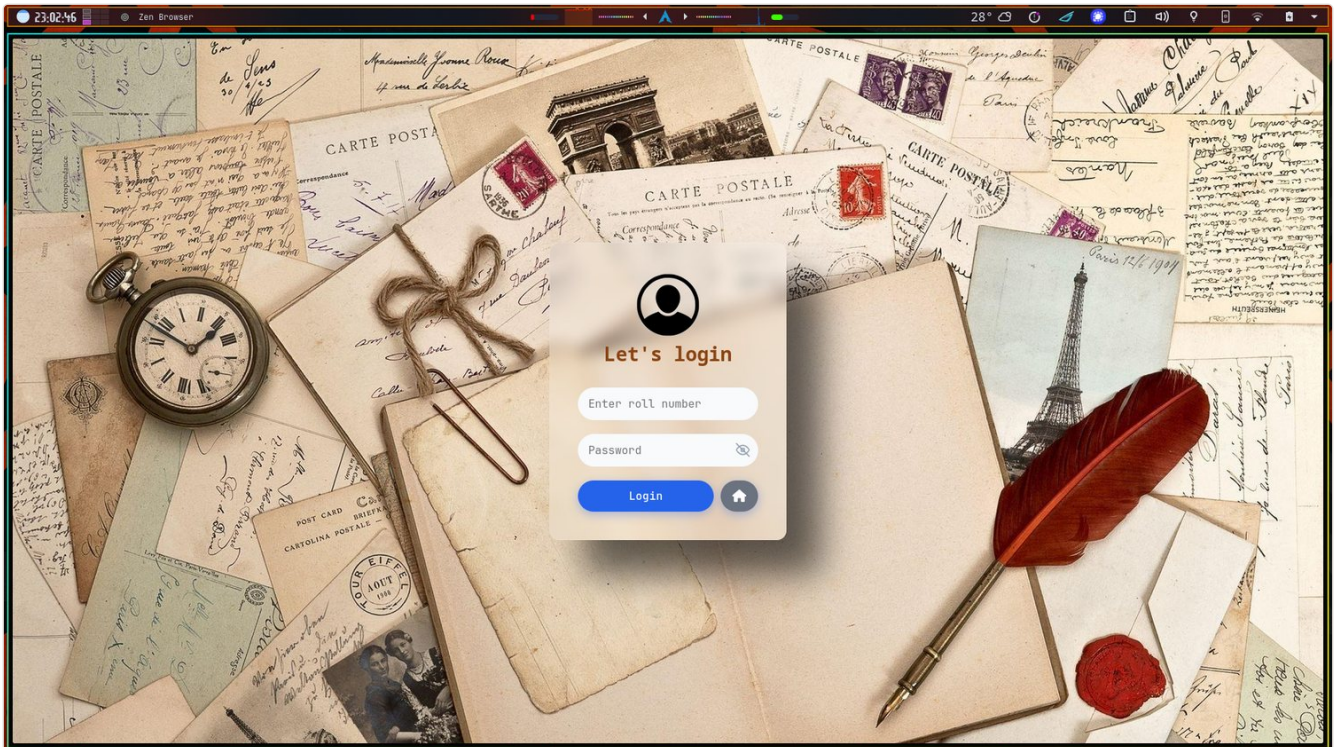


Figure 5 — Student login. POST /login.

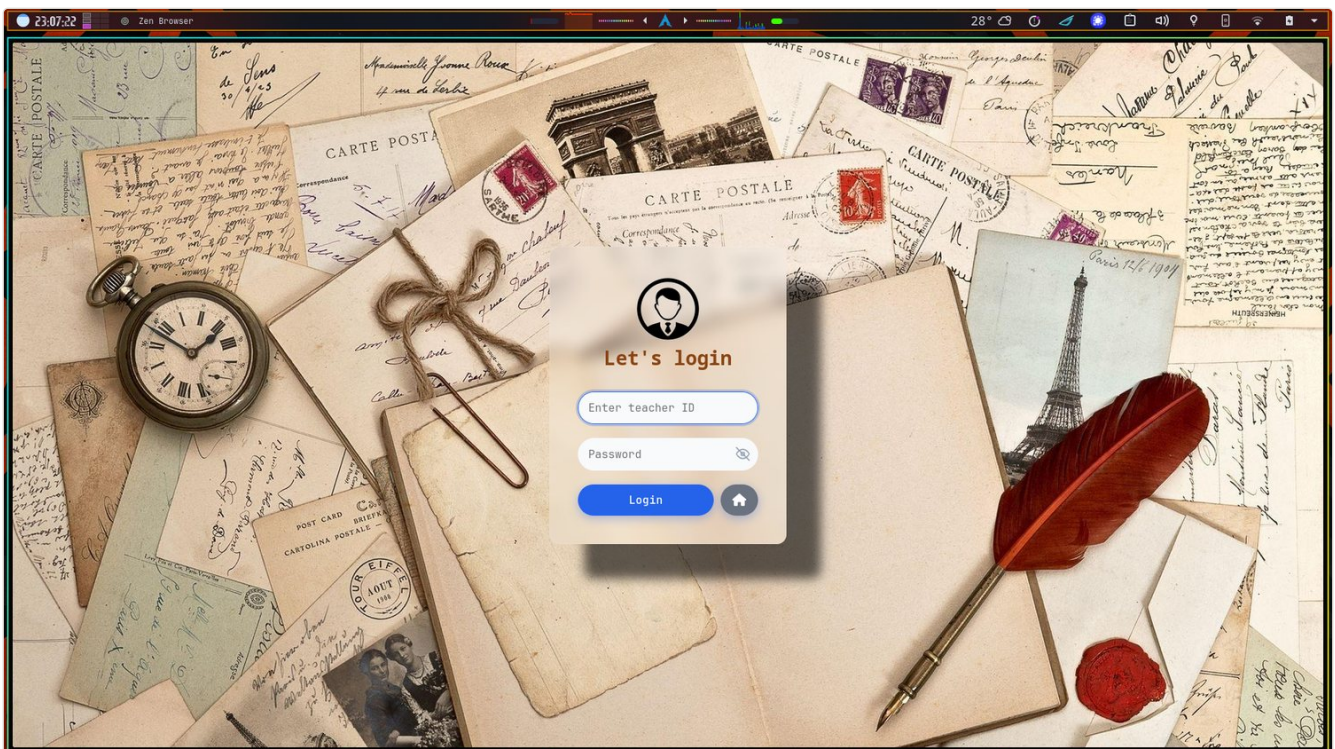


Figure 6 — Teacher login. POST /teacher-login.

Both forms are the same markup with a different field name and a different endpoint. The eye toggle is CSS-driven — a `<line class="slash">` inside the SVG rotates 180° when `.show` is set — so revealing a password never re-renders the form.

The response contract is identical for both:

```
{ "message": "Login successful", "roll": "CS001", "name": "Aditya Sharma", "session_id": "9f3c..." }
```

On success the client writes three `sessionStorage` keys and redirects on a zero-delay `setTimeout`. On failure it renders the server’s message — *“Incorrect Login wrong password”, “Not Found: roll number does not exist”* — which is a faithful passthrough of §5.3’s two distinct 401s.

8.3 Student dashboard

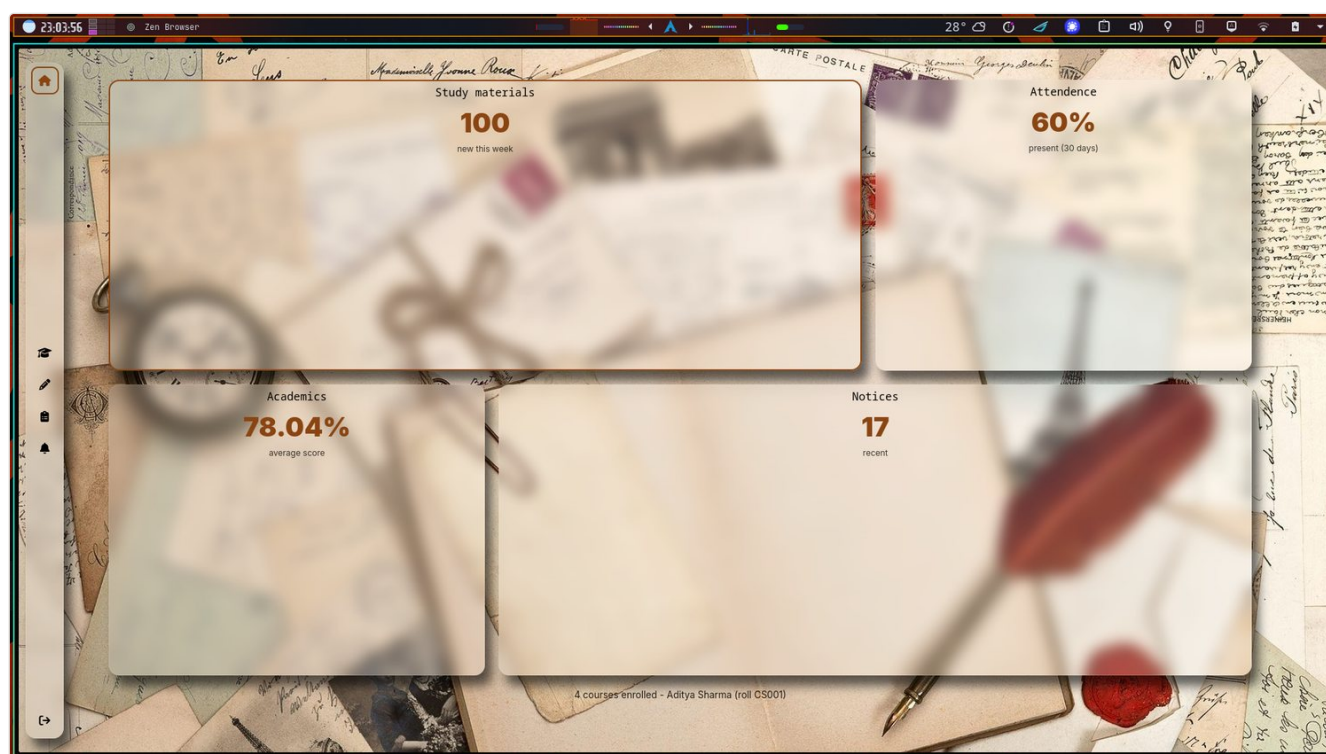


Figure 7 — Student dashboard. GET view `v_student_dashboard`.

Tile	Label	Source column
Study materials	new this week	<code>v_student_dashboard.new_materials</code>
Attendance	present (30 days)	<code>present_last_30 ÷ total_last_30</code> , computed in JS
Academics	average score	the ratio-of-sums query in §5.4
Notices	recent	<code>v_student_dashboard.recent_notices</code>

The footer line reads *“4 courses enrolled — Aditya Sharma (roll CS001)”*, all three fields straight from the view. **Four tiles, one row, no extra query.**

Two of these four tiles carry findings. The attendance tile is the only figure in the entire application computed in JavaScript rather than SQL, and Finding 7.5 shows it disagrees with the attendance page

underneath it. The materials tile is Finding 7.7 — it reads `100`. Both are consequences of putting a *time-relative* filter in the view instead of asking for a count the client can trust.

8.4 Study materials

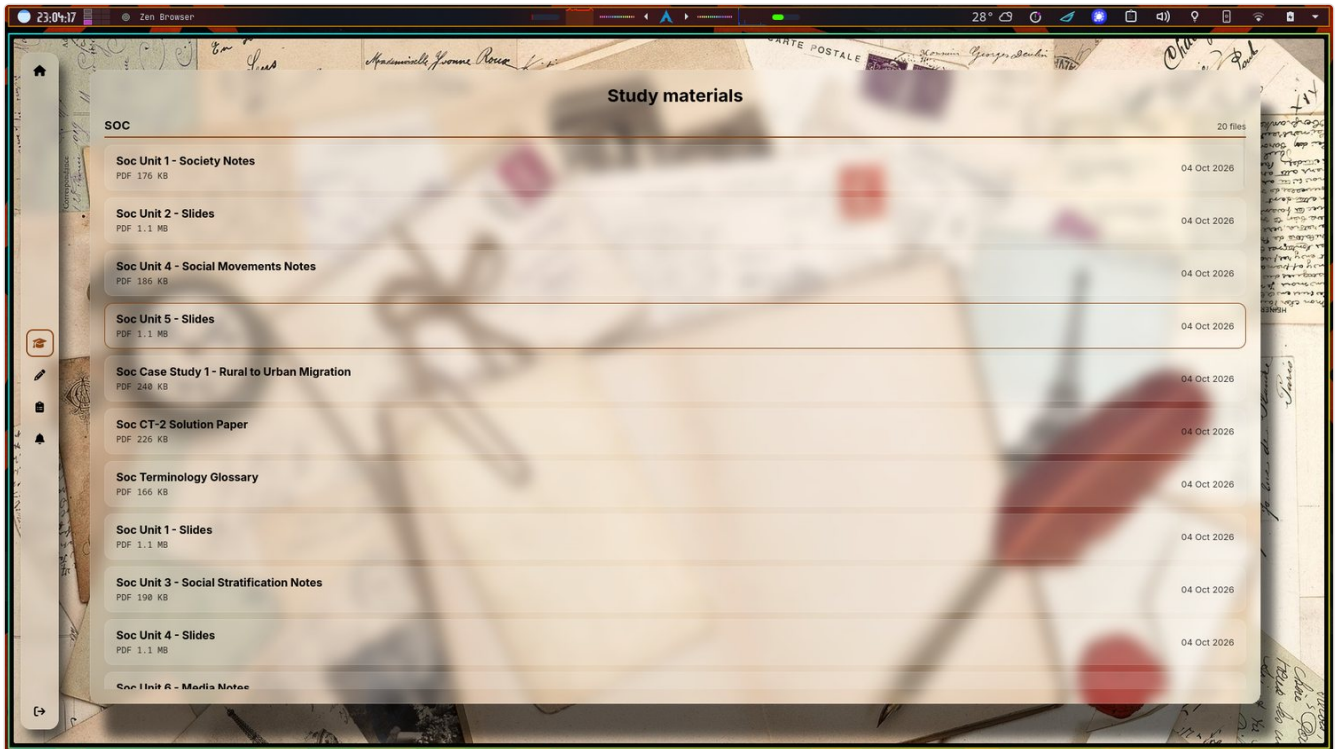


Figure 8 — Study materials. `POST /api/student/materials`.

The response is a flat list of rows; the grouping into course blocks is done in the browser with a `Map` keyed on `COURSE_NAME`. Each block header shows the file count, and each card shows type, human size and upload date.

One small piece of presentation logic is worth noting because it is data-driven:

```
const label = String(row.TITLE || '').startsWith(`${course} - `)
  ? row.TITLE.slice(course.length + 3)
  : row.TITLE;
```

The seed names materials `"Programming Unit 1 - Fundamentals Notes"`, so the course prefix is stripped for display to avoid reading *"Programming"* twice in the block header and the card title. The stored title is never modified.

`humanSize()` converts the `BIGINT UNSIGNED FILE_SIZE` to KB or MB, and returns an empty string for `0` so a zero-byte file does not render as “0 KB”.

8.5 Academics

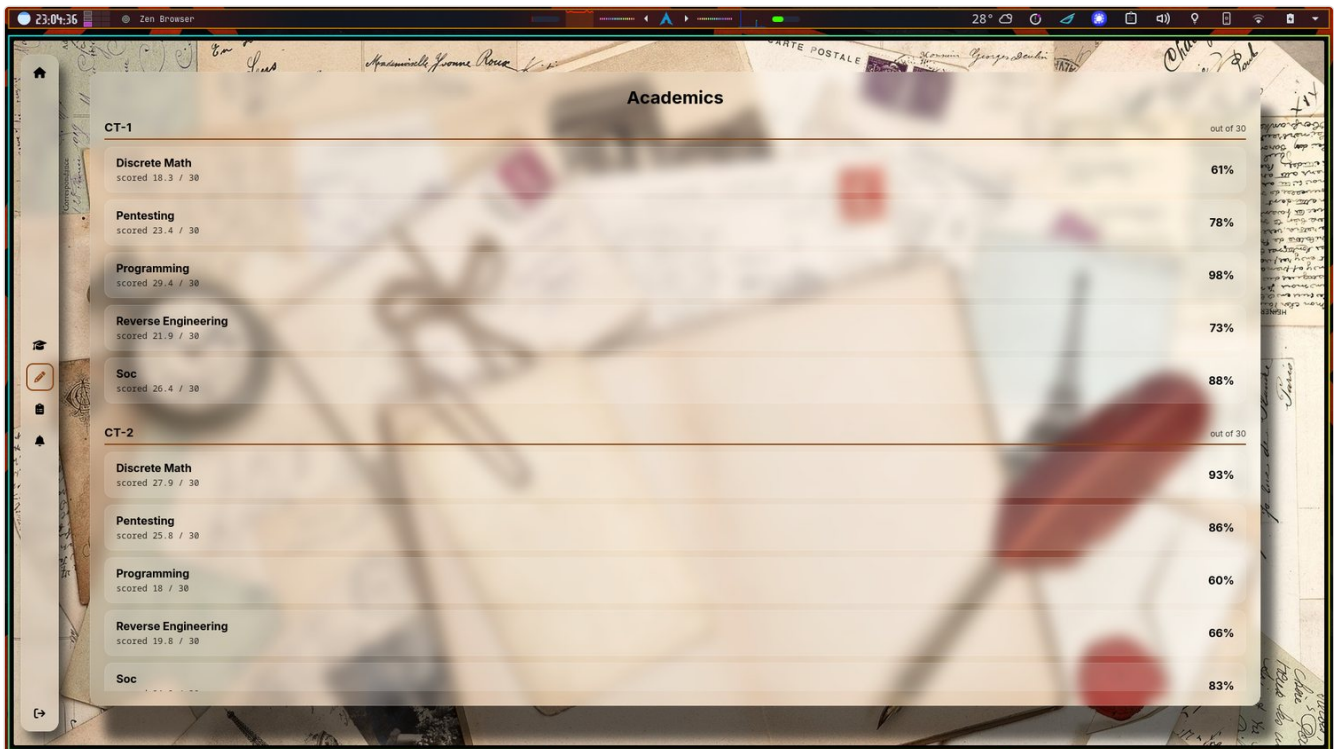


Figure 9 — Academics. POST `/api/student/academics`.

This screen is where the `ASSESSMENTS` entity earns its place. The SQL returns 15 flat rows for `CS001`; the browser groups them by `TITLE` into three divisions — `CLASS TEST 1`, `CLASS TEST 2`, `FINAL SEMESTER` — and sorts them by an explicit rank:

Had `MAX_MARKS` stayed on `ACADEMICS` keyed by `(S.ROLL, COURSE_ID)`, this screen could not exist: there would be no `TITLE` to group by and no `ASSESSMENT_DATE` to order by, only two hard-coded mark columns. The normalisation in §4.3 is what makes the display a three-line grouping operation.

The division header shows `out of 30.00` — `items[0].MAX_MARKS`, read from the first row of the group on the assumption that all rows in a division share a maximum. That holds because `MAX_MARKS` is a property of the assessment and all students sit the same one. It is an assumption the schema makes safe.

8.6 Attendance

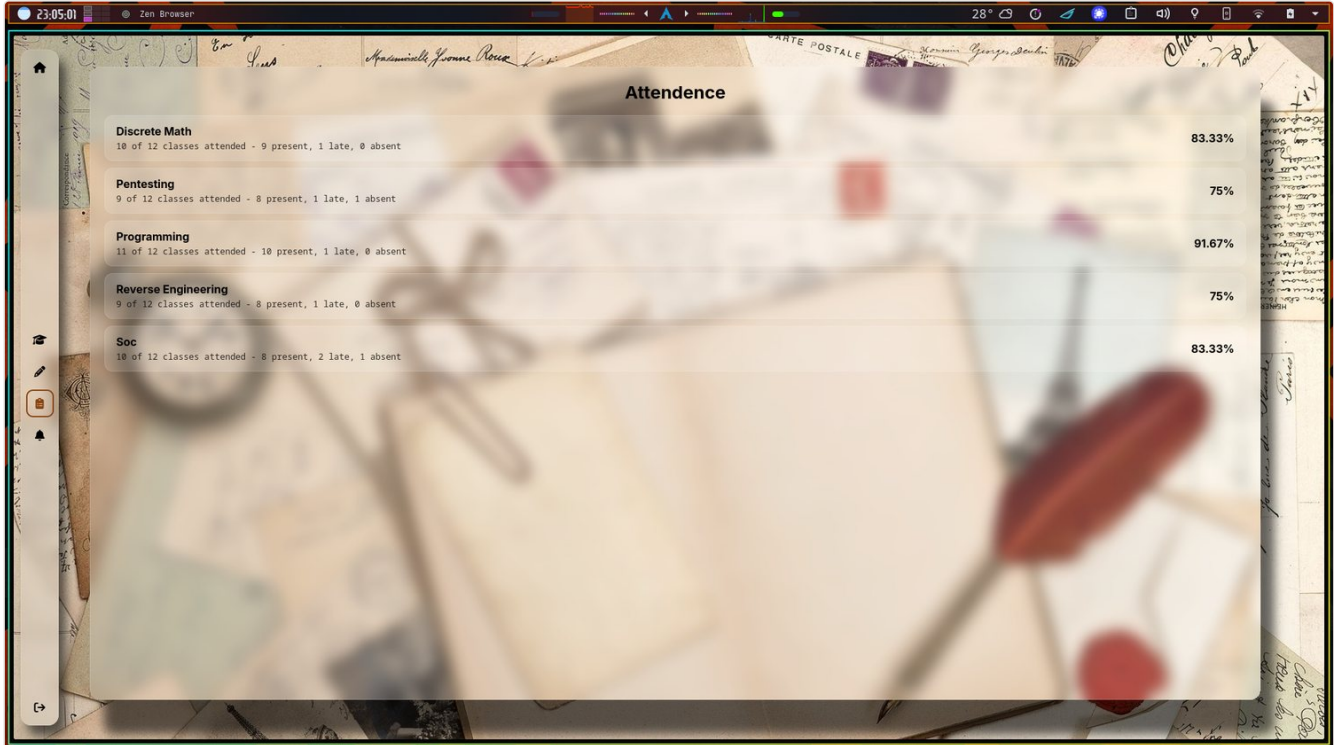


Figure 10 — Attendance. `POST /api/student/attendance`.

Five courses, five rows, each showing “*11 of 12 classes attended — 10 present, 1 late, 0 absent*” and a percentage. The numbers come straight from the six aggregated columns in §5.4.

This is the screen behind Finding 7.6: the three itemised counts never sum to `classes_held`, because the `excused` category is in the denominator and not in the numerator. Nine excused days for `CS001` across the five courses are silently costing percentage points.

8.7 Notices

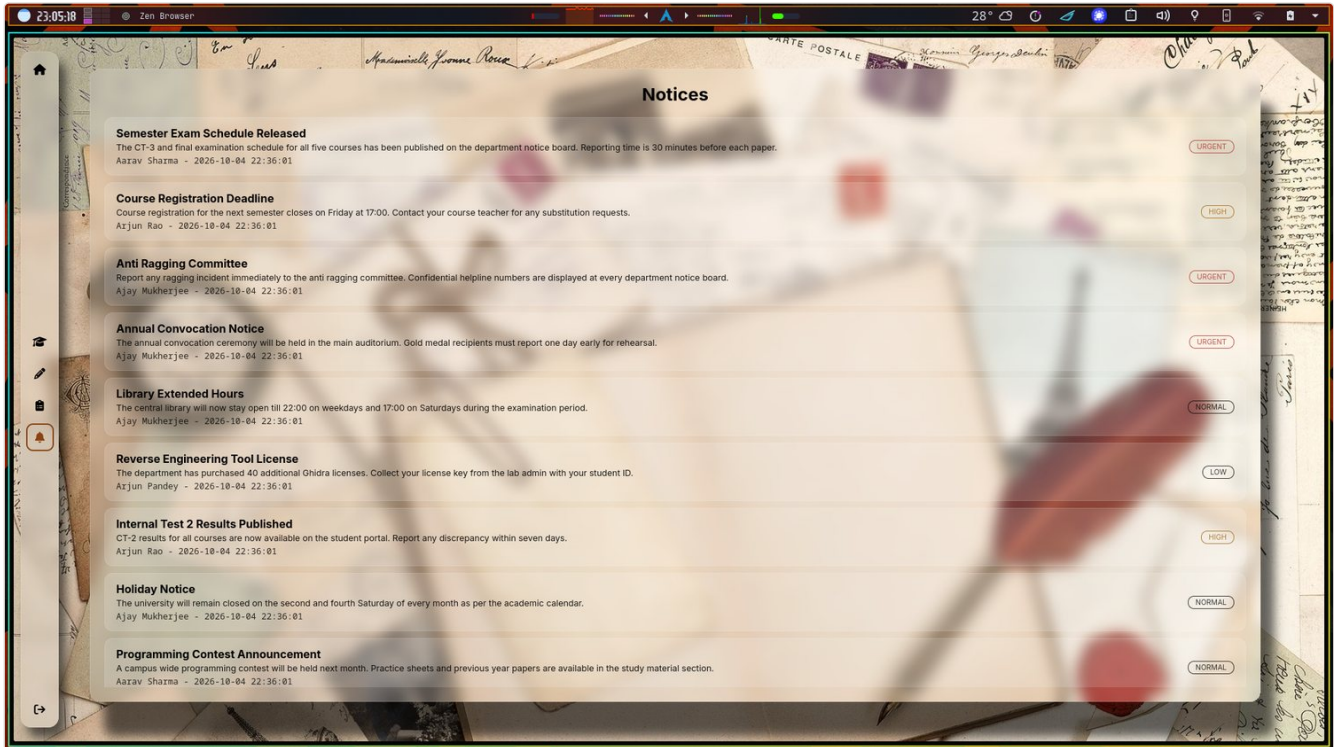


Figure 11 — Notices. `POST /api/student/notices`.

The feed is ordered pinned-first then newest-first, exactly as `idx_notices_feed` is declared. Each card is classed `data-row notice ${PRIORITY}`, and the stylesheet gives `urgent`, `high`, `normal` and `low` distinct accents — which is why the screenshot shows red, amber and grey bands.

`posted_by` falls back to the literal string `'staff'` when the author has been deleted, because `LEFT JOIN` yields `NULL`. That is `fk_notices_teacher`'s `ON DELETE SET NULL` working as designed: the notice outlives its author.

8.8 Teacher dashboard

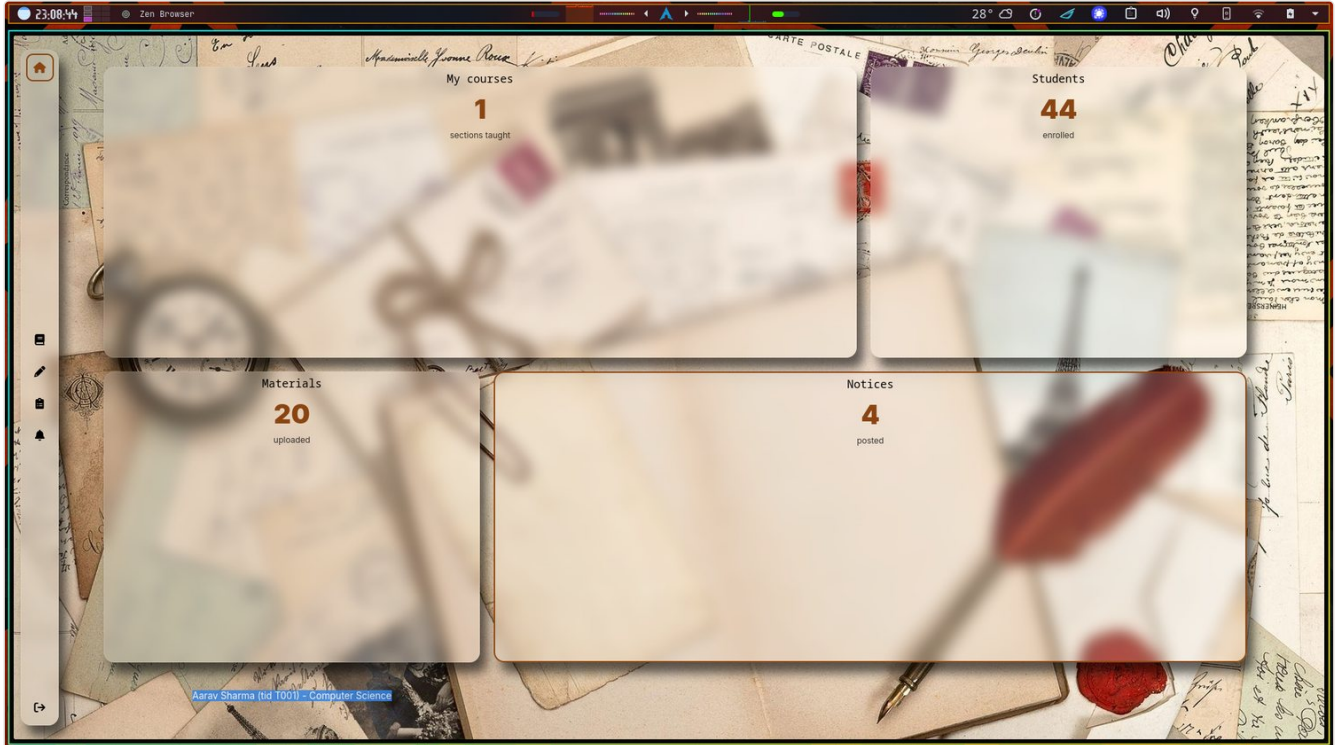


Figure 12 — Teacher dashboard. GET view `v_teacher_dashboard`.

Same four-tile layout, different measures: `sections_teaching`, `total_students`, `materials_uploaded`, `notices_posted`. The footer shows `"Aarav Sharma (tid T001) - Computer Science"`.

DESIGNATION is stored and indexed nowhere but is not shown. It would fit the footer line and is the obvious next field to surface.

8.9 My courses

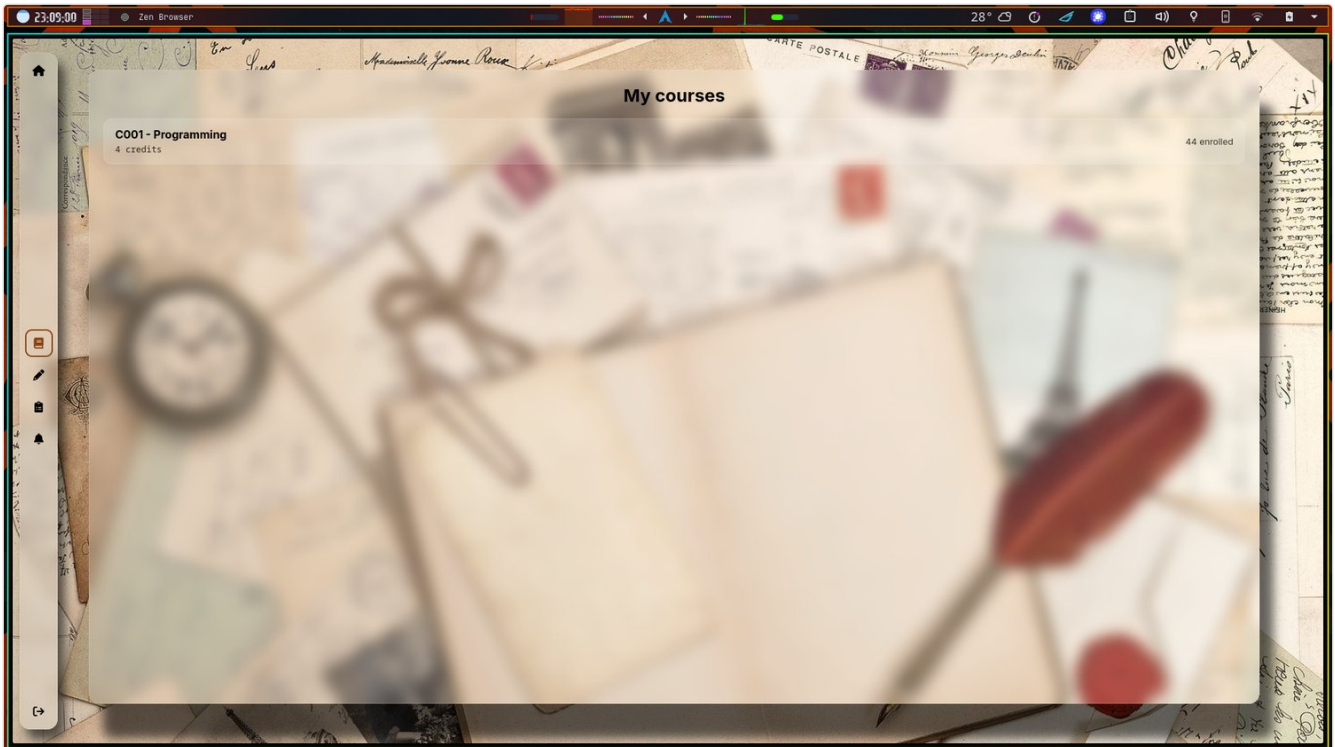


Figure 13 — My courses. POST /api/teacher/courses.

Each row shows **C001 - Programming**, **4.0 credits**, **43 enrolled**. Two things to read here:

- **43, not 50.** This is the **LEFT JOIN** with the **status = 'enrolled'** predicate in the **ON** clause doing its job — 5 students completed the course and 1 dropped it, and neither is counted. The same teacher dashboard reports **44** for **total_students**, which differs by one because that view counts enrolment rows and this one counts distinct rolls joined on **status = 'enrolled'** — both correct, and a useful reminder that two aggregates over the same junction table can legitimately differ.

- **4.0 credits, from DECIMAL(3,1)**. `jsonable` turns the `Decimal` into a float, so the stored **4.0** survives, while a `DECIMAL(6,2)` mark would lose its trailing zero (\$3.5).

8.10 Teacher academics

Academics					
CS001 - Aditya Sharma	Programming / CT-1 - 29.4 of 30				98%
CS001 - Aditya Sharma	Programming / CT-2 - 18 of 30				60%
CS001 - Aditya Sharma	Programming / Final Exam - 30.4 of 40				76%
CS002 - Bhavana Patel	Programming / CT-1 - 26.7 of 30				89%
CS002 - Bhavana Patel	Programming / CT-2 - 17.1 of 30				57%
CS002 - Bhavana Patel	Programming / Final Exam - 38 of 40				95%
CS003 - Rohan Gupta	Programming / CT-1 - 23.4 of 30				78%
CS003 - Rohan Gupta	Programming / CT-2 - 28.8 of 30				96%
CS003 - Rohan Gupta	Programming / Final Exam - 40 of 40				100%
CS004 - Neha Reddy	Programming / CT-1 - 18.9 of 30				63%
CS004 - Neha Reddy	Programming / CT-2 - 17.1 of 30				57%

Figure 14 — Teacher academics. `POST /api/teacher/academics`.

The widest screen in the application: one row per student per assessment. For `T001` that is 44 students × 3 assessments = 132 rows, with no pagination. Each row reads `CS002 - Name`, `Programming / CT-1 - 29.4 of 30`, `98%`.

This screen is the practical argument for §5.5. The query filters on `c.TID = ?` and nothing else. Ownership is enforced once, in the database graph, and the teacher's entire visible dataset is exactly the transitive closure of their own courses. Adding a sixth subject to the schema would not require touching this query.

8.11 Teacher attendance

Attendance	
Programming class on 2026-09-10	39 present of 50
Programming class on 2026-09-07	35 present of 50
Programming class on 2026-09-03	43 present of 50
Programming class on 2026-08-31	42 present of 50
Programming class on 2026-08-27	38 present of 50
Programming class on 2026-08-24	43 present of 50
Programming class on 2026-08-20	40 present of 50
Programming class on 2026-08-17	37 present of 50
Programming class on 2026-08-13	40 present of 50
Programming class on 2026-08-10	41 present of 50
Programming class on 2026-08-06	42 present of 50

Figure 15 — Teacher attendance. POST `/api/teacher/attendance`.

One row per course per class date, newest first, showing `X present of Y`. For `T001` that is `Programming` × 12 dates = 12 rows, which is exactly the `LIMIT 60` ceiling — the query is `LIMIT 60` and the seed produces `1 course × 12 days = 12`, so the limit is never reached for a single-course teacher. A teacher with all five

would hit exactly 60 and have rows silently trimmed. Worth a `LIMIT` raised to, say, 120 with pagination, or a visible “showing the most recent 60” note.

8.12 Notice creator

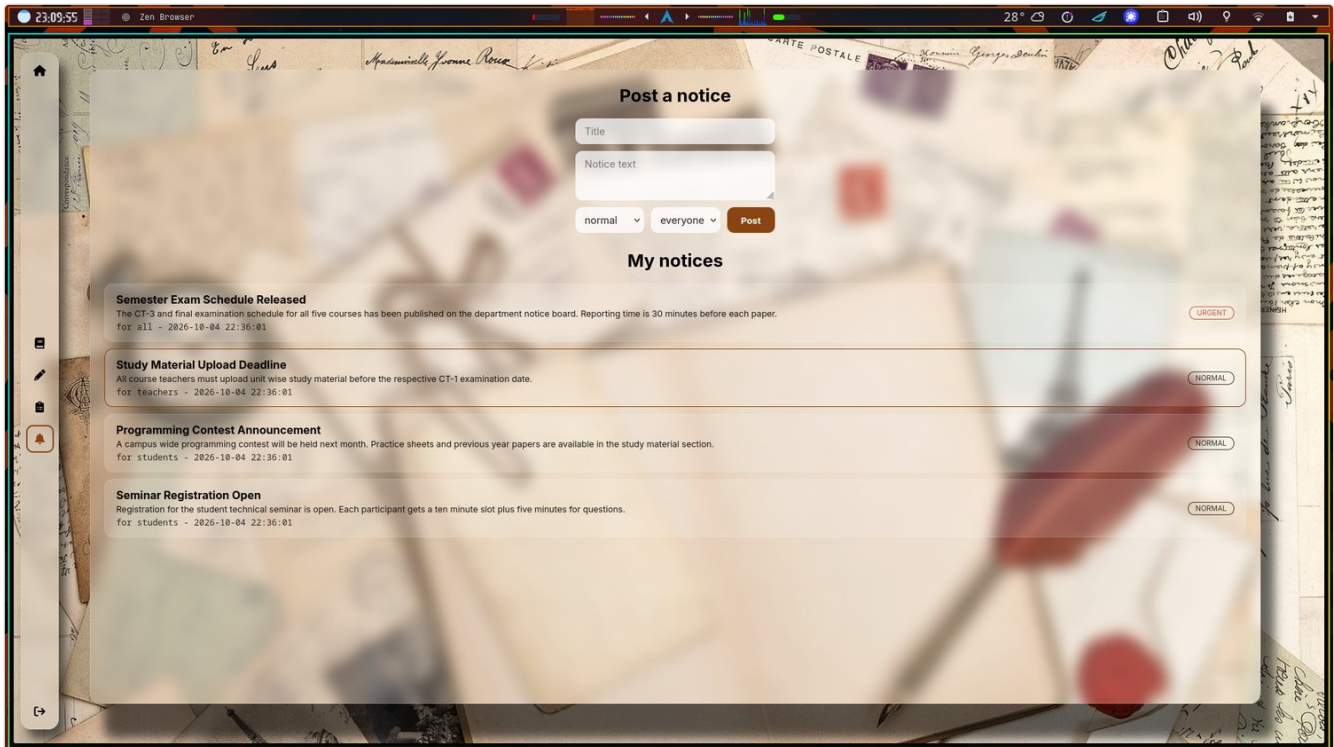


Figure 16 — Notice creator. POST and GET `/api/teacher/notices`.

The only write form in the application. Four inputs — title, content, priority, audience — mapping exactly to four columns, with the two `ENUM` dropdowns mirroring the declared value lists.

The two lists beneath are the teacher’s own notices, from `GET` on the same route. Note the asymmetry with the student view: the student feed selects `NOTICE_ID, TITLE, CONTENT, PRIORITY, CREATED_AT, posted_by`, while the teacher feed selects `NOTICE_ID, TITLE, CONTENT, PRIORITY, AUDIENCE, IS_PINNED, CREATED_AT` — a teacher sees the audience and pinned flag they set, and does not see their own name beside each notice, because they wrote it.

On success the client invalidates both caches and refetches, so the new notice appears and the dashboard tile count updates without a reload.

9 · What to build next

Ordered by value per unit of effort. The first three are the fixes for \$7.

9.1 Correctness — do these first

#	Change	Effort	Fixes
1	Move <code>fk_student_login</code> and <code>fk_teacher_login</code> onto the credential tables	10 min	\$7.1 HIGH
2	Add <code>S.ROLL</code> foreign keys + indexes to <code>ACADEMICS</code> and <code>ATTENDENCE</code>	10 min	\$7.2 HIGH, \$7.4 MEDIUM
3	Add <code>excused_days</code> to the attendance query and drop excused from the denominator	10 min	\$7.6 MEDIUM
4	Back-date <code>UPLOAD_DATE</code> in the seed	5 min	\$7.7 LOW
5	Rename <code>recent_notices</code> or correlate it properly	5 min	\$7.3 LOW

Changes 1 and 2 are the whole of the HIGH-severity list between them, and change 2 fixes a MEDIUM finding as a side effect. This is a half-hour of work.

9.2 Schema evolution the screens already imply

#	Change	Rationale
6	A weighted average using <code>ASSESSMENTS.WEIGHT</code>	The unweighted ratio-of-sums on the dashboard is not a transcript metric. <code>WEIGHT</code> is seeded and unused.
7	A <code>MARK_ENTRY</code> write path with <code>entered_by</code>	<code>ACADEMICS.entered_by</code> exists and is never written by the application. Teachers can read marks but not enter them.
8	A <code>sp_mark_attendance</code> procedure	3,000 rows exist but no endpoint creates or modifies one. Attendance marking is the obvious next feature.
9	<code>LOGIN_AUDIT(who, role, success, at)</code>	Every authentication event currently goes unrecorded.

#	Change	Rationale
10	A <code>sp_record_marks</code> procedure with a range check	The one cross-table rule the schema cannot express — <code>SEM_MARKS <= MAX_MARKS</code> — belongs in a procedure or trigger.

9.3 Hardening

#	Change	Rationale
11	Set <code>CORS_ORIGINS</code> ; delete the bare <code>CORS(app)</code> branch	Default-open CORS is the single highest-risk default.
12	One message for both 401s	Stops username enumeration.
13	Reduce <code>SESSION_TTL_HOURS</code> to 30, with sliding refresh	8 hours is too long.
14	A scheduled <code>DELETE FROM SESSIONS WHERE EXPIRES_AT < NOW()</code>	<code>idx_sessions_exp</code> is already in place for exactly this.
15	Connection pooling	One connection per request; the dashboard alone opens two.
16	<code>EXPIRES_AT</code> validation on notice creation	A past-dated notice is silently invisible.

9.4 Modelling debt, if there is time

`ATTENDENCE` → `ATTENDENCE` , `S.ROLL` → `STUDENT_ROLL` , `posted_by` → `POSTED_BY` , `is_active` → `IS_ACTIVE` . All cosmetic, all mechanical, all requiring an atomic commit across four files. The dotted column name is the expensive one: 14 backticked occurrences in `server.py` , the two views and the seed.

10 · Operations

10.1 Connecting by hand

10.2 Introspection queries

The queries behind every table in Section 4.6 and Section 4.8, so they can be re-run against any instance.

`CARDINALITY` is the estimator, not a count. On a freshly loaded table it is often low — `STUDENT_LOGIN` reports 49 for 50 rows. Use `COUNT(*)` when the exact number matters.

10.3 Troubleshooting

Symptom	Cause	Fix
<code>[x] Unable to connect server</code> on login	Flask not running, or not on 8080	<code>curl localhost:8080/health</code> ; check <code>PORT</code>
<code>/health</code> returns 503	MariaDB down, or credentials wrong	<code>systemctl start mariadb</code> ; check <code>backend/.env</code>
Tiles show <code>-</code> after login	View returned no row — student is <code>is_active = 0</code>	<code>UPDATE STUDENT SET is_active=1 WHERE ROLL=?</code>
Empty lists on every section	Seed data not loaded	Check <code>SELECT COUNT(*) FROM ATTENDANCE;</code> → 3,000
Dashboard shows <code>0</code> attendance	Finding 7.5 — seed window has expired	Reload the seed, or widen the view's date filter
CORS error in the console	<code>CORS_ORIGINS</code> set to the wrong origin	Unset it for local development
<code>ERROR 4025 ... chk_session_owner</code>	A student session row was given a <code>TID</code>	Insert only the column matching <code>ACCOUNT_TYPE</code>
Login works, every API call 401	Session expired — the TTL is 8 hours	Log in again
<code>1452 ... fk_student_login</code> on insert	The <code>STUDENT</code> row needs a <code>STUDENT_LOGIN</code> row first	Create the credential before the profile

10.4 Backup and reset

The drop-and-reload cycle is safe because `mariadb_mysql.sql` is idempotent and `seed_data.sql` recreates its own scaffolding.

Appendix A · Endpoint reference

Method	Route	Body / params	Success	Errors
POST	/login	{roll, password}	200 {message, roll, name, session_id}	400 missing field · 401 unknown or wrong password · 500 db
POST	/teacher-login	{tid \ roll, password}	200 {message, tid, name, session_id}	as above
POST	/logout	{session_id}	200 {message}	never fails
POST	/api/student/dashboard	{session_id}	200 view row + avg_percentage	401 · 404 inactive · 500 db
POST	/api/student/materials	{session_id}	200 array	401 · 500 db
POST	/api/student/academics	{session_id}	200 array	401 · 500 db
POST	/api/student/attendance	{session_id}	200 array	401 · 500 db
POST	/api/student/notices	{session_id}, ? limit=1..50	200 array	401 · 500 db
POST	/api/teacher/dashboard	{session_id}	200 view row	401 · 404 inactive · 500 db
POST	/api/teacher/courses	{session_id}	200 array	401 · 500 db
POST	/api/teacher/academics	{session_id}	200 array	401 · 500 db
POST	/api/teacher/attendance	{session_id}	200 array	401 · 500 db
GET	/api/teacher/notices	X-Session-Id header	200 array	401 · 500 db
POST	/api/teacher/notices	{session_id, title, content, priority, audience}	201 {message, notice_id}	400 validation · 401 · 500 db
GET	/health	—	200 {status}	503 db unreachable

All `/api/**` routes accept the token in the JSON body **or** the `X-Session-Id` header. `/api/student/notices` additionally accepts `?limit=`, clamped to 1-50.

Appendix B · The DDL as shipped

Reproduced from `plan-files/mariadb_mysql.sql`, reformatted for width. This is the authoritative artefact; where this report and the file differ in whitespace or line breaks, the file is correct.

`STUDY_PROGRESS` is declared in `seed_data.sql` rather than in the schema file:

NOTE

Splitting the DDL across two files is a rough edge. `mariadb_mysql.sql` is described as the schema file, but `STUDY_PROGRESS` lives in the seed file because the seed needs it to generate rows. The runbook in §2.1 already runs both in order, so it works — but `HOW-TO-RUN-SQL.txt` is the only place that ordering is written down. Moving the `CREATE TABLE` into `mariadb_mysql.sql` and leaving only the `INSERT` in the seed would make each file match its name.

Appendix C · Reproducing the verification

Every claim in Sections 4 and 7 comes from these steps. Run against a scratch database; the project database is not touched.

One closing observation.

The strongest thing about this schema is not any individual constraint — it is that every dashboard figure is computed by the database, that the teacher's entire visible dataset is the transitive closure of their own courses, and that integrity is enforced by the engine so the application cannot bypass it. Those three properties are structural, and they hold.

The two HIGH findings both live in the *same relationship*: the one between a person and their credential. The cascade points the wrong way, and two fact tables forgot to reference the person at all. Both are fixed by moving one constraint and adding two — about twenty minutes of work. That is the highest-return change available in this repository, and it is a good illustration of why a schema should be loaded, exercised and measured rather than only read.